Linköping University | Department of Computer and Information Science

Master's thesis, 30 ECTS | Datateknik

2026 | LIU-IDA/LITH-EX-A--2026/001--SE

LLM-assisted SOC Alert Analysis : A Design Space Evaluation of system variants

LLM-assisterad analys av SOC-larm: En utvärdering av designrymden för systemvarianter

Vincent Garbrant
Hannes Lundberg

Supervisor : Cajsa Wargren
Examiner : Fredrik Heintz

LINKÖPING UNIVERSITY

Linköpings universitet
SE–581 83 Linköping
+46 13 28 10 00 , www.liu.se

Upphovsrätt

Detta dokument hålls tillgängligt på Internet - eller dess framtida ersättare - under 25 år från publiceringsdatum under förutsättning att inga extraordinära omständigheter uppstår.

Tillgång till dokumentet innebär tillstånd för var och en att läsa, ladda ner, skriva ut enstaka kopior för enskilt bruk och att använda det oförändrat för ickekommersiell forskning och för undervisning. Överföring av upphovsrätten vid en senare tidpunkt kan inte upphäva detta tillstånd. All annan användning av dokumentet kräver upphovsmannens medgivande. För att garantera äktheten, säkerheten och tillgängligheten finns lösningar av teknisk och administrativ art.

Upphovsmannens ideella rätt innefattar rätt att bli nämnd som upphovsman i den omfattning som god sed kräver vid användning av dokumentet på ovan beskrivna sätt samt skydd mot att dokumentet ändras eller presenteras i sådan form eller i sådant sammanhang som är kränkande för upphovsmannens litterära eller konstnärliga anseende eller egenart.

För ytterligare information om Linköping University Electronic Press se förlagets hemsida `http://www.ep.liu.se/`.

Copyright

The publishers will keep this document online on the Internet - or its possible replacement - for a period of 25 years starting from the date of publication barring exceptional circumstances.

The online availability of the document implies permanent permission for anyone to read, to download, or to print out single copies for his/hers own use and to use it unchanged for non-commercial research and educational purpose. Subsequent transfers of copyright cannot revoke this permission. All other uses of the document are conditional upon the consent of the copyright owner. The publisher has taken technical and administrative measures to assure authenticity, security and accessibility.

According to intellectual property law the author has the right to be mentioned when his/her work is accessed as described above and to be protected against infringement.

For additional information about the Linköping University Electronic Press and its procedures for publication and for assurance of document integrity, please refer to its www home page: `http://www.ep.liu.se/`.

© Vincent Garbrant
Hannes Lundberg

Abstract

Analysts in Security Operations Centers (SOCs) face large volumes of security alerts, many of which require repetitive contextual investigation before they can be classified as malicious or benign. Large language model (LLM) agents offer a possible way to automate parts of this triage work, but it is unclear which contextual factors such agents need and how these factors should be exposed to avoid misleading conclusions. This thesis investigates how an LLM-based agent can support selected aspects of SOC alert analysis by combining alert data, surrounding evidence, and structured investigation guidance.

The study designs and evaluates a SOC alert-analysis pipeline in a controlled SOC-like environment. The pipeline is based on contextual factors from prior work on how SOC analysts investigate alerts, especially the factors described by Kersten et al. These factors were implemented through Model Context Protocol (MCP) tools, forming the Kersten-factor MCP (K-MCP) configuration, and refined through prompt changes, tool-description changes, and progressive disclosure of evidence. The evaluation uses a labelled alert set that combines benign alerts generated in the lab environment with malicious alerts taken from the public MORDOR 2019 dataset. Verdict correctness is evaluated with F1, precision, and recall, while report quality is evaluated with MESSALA, an LLM-based report-quality evaluation framework.

In this controlled evaluation, the results show that the way contextual evidence is given to the agent has a clear effect on performance. A final classification performance was summarized using three repeated runs of the selected scoped K-MCP configuration. This configuration, which excluded alert history and surrounding-alert context to reduce contextual overreach, averaged F1 = 0.942 ± 0.028 on the 76-alert evaluation set. The base reference result achieved F1 = 0.711 ± 0.016. The improvement mainly came from reducing false positives on benign alerts: the base result produced 12 false positives, while the selected scoped K-MCP configuration produced an average of 1.0 false positives. Both configurations detected 16 of 17 malicious alerts. MESSALA complemented the verdict metrics by showing how the contextual variants affected the quality and grounding of the generated reports.

These findings suggest that, in a controlled SOC-like evaluation setting, LLM agents can be studied as evidence-guided decision-support systems rather than as standalone classifiers. The contribution of this thesis is not to show that the system is ready for operational use. Instead, the thesis studies how selected contextual factors and tool-design choices affect an LLM-assisted alert-analysis pipeline. The results also show that context must be scoped carefully: adding more contextual information is not always beneficial and can increase misleading reasoning.

Contents

Abstract iii

Acknowledgments iv

Contents iv

List of Figures vi

List of Tables vii

1 Introduction 1
- 1.1 Motivation . . . 1
- 1.2 Aim . . . 2
- 1.3 Approach . . . 2
- 1.4 Research questions . . . 2
- 1.5 Delimitations . . . 2
- 1.6 Contributions . . . 2
- 1.7 Structure of the Thesis . . . 3

2 Background 4
- 2.1 Security Operations Centers and Alert Triage . . . 4
- 2.2 Benign Alerts, False Positives, and Background Noise . . . 6
- 2.3 Information Needed for Alert Triage . . . 6
- 2.4 Transformers and Attention . . . 8
- 2.5 Large Language Models . . . 8
- 2.6 Chain-of-Thought Prompting . . . 8
- 2.7 LLM Agents and Tool Use . . . 9
- 2.8 Model Context Protocol . . . 9
- 2.9 Evaluation of SOC Analysis Reports . . . 9
- 2.10 ELK Stack . . . 10

3 Related Work 11
- 3.1 LLMs for Security Log Analysis . . . 11
- 3.2 AI Support for SOC Analysts . . . 11
- 3.3 LLM Agents in Cybersecurity . . . 12

4 Method 13
- 4.1 Research Design and RQ Mapping . . . 13
- 4.2 Lab Environment . . . 14
- 4.3 Data Generation . . . 14
- 4.4 Data Collection and Storage . . . 17
- 4.5 Benign Alert Retrieval and Sampling . . . 18
- 4.6 Malicious Alert Reference Set . . . 18

4.7 Prototype Implementation . . . 18
4.8 Operationalizing Kersten Factors Through MCP . . . 21
4.9 Development Iterations and Final Configuration . . . 23
4.10 System Variants . . . 24
4.11 Evaluation Design . . . 25

5 Results **34**
5.1 Overview of Results . . . 34
5.2 Results for Research Question 1 . . . 34
5.3 Results for Research Question 2 . . . 37
5.4 Results for Research Question 3 . . . 38
5.5 Summary of Findings . . . 39

6 Discussion **40**
6.1 Overview of the Discussion . . . 40
6.2 Discussion of Research Question 1 . . . 40
6.3 Discussion of Research Question 2 . . . 42
6.4 Discussion of Research Question 3 . . . 44
6.5 Methodological Discussion . . . 45
6.6 The Work in a Wider Context . . . 45
6.7 The recurring malicious alert assessed as benign . . . 46
6.8 Limitations and Future Work . . . 46
6.9 Summary of the Discussion . . . 47

7 Conclusion **48**

Bibliography **50**

A JSON Scripts Used for Data Generation **54**

List of Figures

2.1 Simplified relationship between log events, detection logic, and an alert. The alert starts the triage process but does not contain all evidence needed for the final assessment. 5

4.1 Testbed architecture detailing the monitoring and orchestration routing between Host A and Host B. 15
4.2 Role separation in the LangChain ecosystem for this study. LangChain provides reusable components, LangGraph orchestrates execution, and LangSmith supports tracing and evaluation. 19
4.3 Shared LangGraph structure and differing analysis-node internals for the base agent and K-MCP agent. 20
4.4 Example of contextual overreach in a diagnostic K-MCP report. The agent used surrounding or historical alerts as incriminating evidence against the focal alert, even though such alerts are only contextual signals unless they provide direct support for the focal event. 24
4.5 Construction of the MESSALA knowledge base from Kersten et al.'s four-stage process, SOC-practitioner interviews, and IT/OT reporting literature. 28
4.6 Evaluation pipeline for generated alert reports. The report-quality path selects MESSALA checklist items c_i, scores each item as m_i, and aggregates the resulting scores into $e_M(R)$, while the classification path extracts $v(R)$ as $\hat{y}$ and compares it with the ground-truth label y to calculate e_F. 29
4.7 Item-level MESSALA scoring using parallel high-level and in-depth perspectives followed by multi-perspective reconciliation. The final per-item result contains the numerical score, explanation, evidence gaps, and actionable feedback returned to the report-level evaluation. 30
4.8 Overall alert-report evaluation workflow. Alert information is provided to the LLM agent, while the ground-truth label y remains hidden until the extracted prediction $\hat{y}$ is compared with it for e_F scoring. The report-quality metric e_M is computed separately with MESSALA. 33

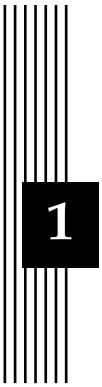
6.1 Report excerpts for the Related logs ablation. The focal scheduled-task evidence is the same in both variants, but removing Related logs changes the interpretation from administrative or helpdesk testing activity to suspicious reconnaissance or staging. 43

List of Tables

2.1 Kersten-derived factors relevant to alert triage. 7
2.2 Categories of information SOC analysts employ to classify alerts 7

4.1 GHOSTS Administrative Timeline Partials and Literature Grounding 16
4.2 GHOSTS HR and Office Timeline Partials and Literature Grounding 17
4.3 Overview of the evaluation components. 26

5.1 Historical development runs used to guide final configuration selection. 35
5.2 Classification performance for the original systems. 35
5.3 Base agent and selected scoped K-MCP performance on the 76-alert evaluation set. 35
5.4 Clean repeated runs of the selected scoped K-MCP configuration. 36
5.5 Qualitative comparison of two reports for the same benign scheduled-task alert. . 36
5.6 Formatted summary of the good and bad report examples for the same benign alert. 36
5.7 Selected report excerpts from the good and bad examples. 37
5.8 Selected MESSALA feedback for the good and bad report examples. 37
5.9 Repeated ablation-condition summary. 37
5.10 MESSALA summary relative to the selected scoped K-MCP condition. 38



1 Introduction

As digitalization continues, cybersecurity is becoming an increasingly important issue for modern societies. More industries are moving their infrastructure online, which presents a growing set of targets for malicious actors. One emerging concern is the increasing alert volume faced by security operations centers (SOCs).

1.1 Motivation

A SOC's primary function is to monitor network activity, handle alerts, and investigate the events that cause alerts to trigger. Everyday activity in an enterprise network produces substantial amounts of logs, making contextual information essential for effective triage. In modern SOCs, rule-based systems trigger alerts that are then analyzed by security analysts. These systems classify alerts into different severity levels. A large share of alerts are low-level, and only a fraction of alerts lead to incidents; in other words, many alerts are false positives. This high volume of false positives causes analysts to spend substantial time on alerts that are likely benign. One resulting phenomenon is alert fatigue, where workers become desensitized to alerts [22, 39]. A key contributor to alert fatigue is the high volume of false positives presented to analysts [1]. This can result in alerts not being thoroughly investigated and can diminish alert-response capability, increasing the risk that true positives slip through the SOC.

To increase SOC efficiency and reduce analyst fatigue, it is crucial to filter out benign alerts and reduce alert investigation time. Because logged event sequences vary and can be complex, adjusting rule-based alert systems to cover all benign and malicious variations is, for practical purposes, not feasible. Recent developments in AI and machine learning may help alleviate this problem. LLMs could reduce the analysis time required per alert by summarizing alert evidence and surrounding context. If LLMs can support analysts in handling high alert volumes, SOCs could improve efficiency and reduce operational costs.

1.2 Aim

The primary aim of this thesis is to investigate how LLMs can potentially improve the efficiency and effectiveness of a security analyst's triage process by providing contextualized investigation summaries. The thesis first examines AI methods applicable to the problem and reviews related work to establish the project's theoretical foundation. Based on this foundation, the work iteratively develops an LLM-based agent pipeline that contextualizes, summarizes, and investigates alert information to produce a SOC report. The goal is for the LLM output to resemble a report produced by SOC analysts in industry. This thesis also identifies key factors that affect a SOC analyst's decision process for an alert, drawing on literature about SOC analyst work. During pipeline development, the design is guided by this literature base. The underlying hypothesis is that an agent designed around important components of SOC analyst work is expected to perform well. This thesis also studies how each key factor affects the agent's performance to identify which types of information are essential for LLM-assisted SOC triage.

1.3 Approach

This thesis uses an experimental research design based on iterative development. The study starts with a base LLM configuration, which is tested to identify missing factors that lead to insufficient analysis. Identified factors are then incorporated through layered expansion of the model's external capabilities, ranging from prompt-level adjustments to agentic architecture and MCP-based tools.

1.4 Research questions

1. How do contextual tool access and prompt-level design choices affect the performance of an LLM-assisted SOC alert-analysis pipeline compared with a base LLM agent?
2. How do different *Kersten* factors contribute to the performance of the LLM-assisted SOC alert-analysis pipeline?
3. How can a controlled benign dataset be created for evaluating LLM-assisted alert triage?

1.5 Delimitations

- Primary focus on summarization and contextualization rather than automated remediation.
- Evaluation using historical and simulated data rather than live production deployment.
- Evaluation limited to selected quantitative and qualitative metrics rather than comprehensive organizational impact.
- Focus on different LLM configurations rather than different models; therefore, the analysis is limited to a single model.

1.6 Contributions

The main contributions of this thesis are threefold. First, the thesis presents the design and implementation of an LLM-assisted pipeline for SOC alert analysis, structured to support alert triage through configurable contextual capabilities. Second, it provides a comparative

evaluation of system variants to study how different factor configurations affect verdict quality and report quality. Third, it contributes a controlled evaluation environment and a benign alert dataset with surrounding background noise, enabling reproducible assessment of LLM-assisted alert analysis in a context that is closer to practical SOC work than isolated alert records alone.

1.7 Structure of the Thesis

The remainder of the thesis is organized as follows. Chapter 2 presents the background relevant to SOC alert analysis, contextual triage, and LLM-based support systems. Chapter 3 reviews related work. Chapter 4 describes the methodological approach, including the lab environment, data generation, and evaluation design. Chapter 5 presents the empirical findings, which are then interpreted in Chapter 6. Finally, Chapter 7 concludes the thesis and summarizes its main findings and contributions.

2 Background

This chapter gives context to the concepts needed to understand the thesis. It aims to provide necessary information to understand the following chapters. Security operations center, alerts and LLMs are amongst the topics that are explained.

2.1 Security Operations Centers and Alert Triage

The SOC emerged to centralize security operations and address the increasing volume and cost of cyberattacks. While these centers initially functioned as reactive network monitoring units, they have transitioned toward more holistic "Security Intelligence Centers" to gain an integrated view of threats [43].

Security operations centers face a major operational bottleneck known as "alert fatigue" [39]. Alert fatigue is described as a multifaceted issue driven by high false-positive rates, a chronic shortage of skilled staff, and the cognitive load caused by disconnected and overloaded dashboards [39]. The emphasis is on the difficulty of SOCs in keeping up with the volume and variety of alerts.

The operational consequence of alert fatigue is a reduced ability to effectively prioritize threats, resulting in missed indicators of compromise or analyst burnout. The industry has identified AI-assisted triage as a primary research opportunity to mitigate these pressures [39]. By integrating AI to handle the initial investigation and contextualization of events, the goal is to reduce the manual time required to triage and ensure that analysts focus on high-value investigations.

A traditional SOC environment relies heavily on analysts' capacity to handle a multitude of alerts. When an alert is received in a **Security Information and Event Management** (SIEM) system, an analyst starts by assessing the alert to separate a false positive from an actual threat [39].

To combat the issue of alert fatigue in SOC environments, recent literature has focused on implementing AI and machine learning as an aid [6, 16].

Figure 2.1 shows triage as the first assessment step that follows when a monitoring system or SIEM raises an alarm. During this step, the analyst determines whether the alert is likely benign or whether it indicates malicious activity. Since triage sits between large-scale monitoring and full incident response, it becomes one of the central points in daily SOC work [43, 39].

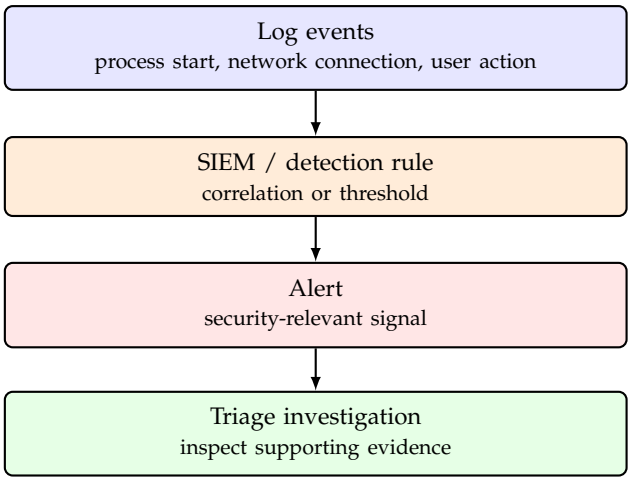


Figure 2.1: Simplified relationship between log events, detection logic, and an alert. The alert starts the triage process but does not contain all evidence needed for the final assessment.

In practice, alert triage is often performed by Tier-1 analysts, whose task is to gather enough evidence to support an operational decision within a limited amount of time. Rather than relying on the alert alone, analysts combine information from alert metadata, related logs, host and network context, and knowledge about the monitored environment [21, 20]. This makes alert triage a contextual reasoning task since the same technical indicator can be suspicious in one environment and benign in another.

The importance of context is also emphasized in qualitative studies of SOC work. Alahmadi et al. show that analysts rely on tacit knowledge and an understanding of normal behavior when deciding whether an alarm is meaningful [1]. Additionally, high false-positive rates, overloaded dashboards, and time pressure make it difficult to investigate each alert thoroughly, which contributes to alert fatigue in SOCs [1, 39]. This combination of high operational relevance and strong dependence on contextual evidence makes alert triage an important setting for AI-assisted SOC support [39, 6].

Alerts and Log Events

An alert is the output produced by a SIEM, detection rule, or correlation mechanism when observed activity matches a condition that has been defined as security-relevant. The alert is therefore an event that points to potentially suspicious activity, and not the raw activity itself. A log event is a timestamped record of activity from a source such as a host, process, network connection, application, authentication system, or user action. SIEM systems collect these events from multiple sources and use rules or correlation logic to produce alerts when relevant event patterns are observed [29, 14].

One alert can be based on a single log event, but it is often created from several related events that together satisfy a rule or correlation condition. For that reason, the alert should be treated as the starting point of the investigation rather than as the full investigation. The analyst still needs to inspect the underlying and surrounding evidence, such as related logs, host

behaviour, earlier activity, and other contextual information, before deciding whether the alert represents benign or malicious activity [21, 20, 1].

Elastic Alert Fields

An Elastic alert is a structured record that contains information about the event that triggered a detection rule. In these alert logs, certain fields are used to sort the information about the event into a clear and structured view. Examples of these fields include:

- `@timestamp` shows when the alert occurred.
- `host.name` identifies the machine where the event occurred.
- `user.name` identifies the involved user related to the event.
- `process.command_line` shows the executed command line when process details are available.

It is important to note that there are different alert log types depending on the potentially malicious behavior, which means that the Elastic alert fields also vary. These fields can be useful for initial assessment and act as guidance when conducting the investigation, but deeper investigation with other logs might be required to conclude a scenario as benign or malicious.

2.2 Benign Alerts, False Positives, and Background Noise

Benign alerts are alerts triggered by activity that matches a suspicious pattern or detection rule, but which is assessed to originate from non-malicious behaviour. Such alerts may be caused by ordinary user activity, administrative actions, software behaviour, or other legitimate events that resemble attack techniques in the available logs. This distinction is important because an alert cannot be judged only from the rule match itself; analysts often need surrounding evidence of the event before deciding that the activity is harmless [1, 21, 20].

A false positive is an alert that satisfies the conditions of a detection rule and is therefore presented as suspicious, but which is later assessed to be benign. In SOC environments, false positives are problematic because they increase the number of alerts analysts must inspect without necessarily indicating real malicious activity. High false-positive rates take time from analysts, contribute to alert fatigue, and make it harder to prioritize alerts that represent actual threats [1, 39].

An important distinction is that benign activity does not always look benign in the alert logs. Administrator actions, user activity, software updates, and misconfigurations can all produce events that appear suspicious at first. Because of this, analysts do not base their assessment only on the alert itself. They also look at related logs, host information, previous activity, and other surrounding events to understand why the alert was triggered [21, 20]. This surrounding data can be large and noisy. Finding the relevant parts is therefore an important part of triage, and one reason why false positives and alert fatigue remain difficult problems in SOC work [1, 39].

2.3 Information Needed for Alert Triage

Several studies indicate that successful alert triage depends on more than the alarm signature itself. Alahmadi et al. [1] report that the practical value of an alert is shaped both by the quality of the detection rule and by the analyst's access to supporting context. Poorly written

signatures can generate such large volumes of alarms that analysts do not have time to review them, while missing environmental context makes it harder to distinguish benign activity from genuine threats [1]. In other words, useful triage depends both on manageable alert quality and on the availability of surrounding evidence.

Kersten et al. provide a more structured description of what this surrounding evidence looks like in Tier-1 investigations. In their synthesis of a threat analysis process for SOC analysts, they group the information used during alert classification into four categories: relevance indicators, additional alerts, contextual information, and attack evidence [21]. In their later field study, they show that contextual information is the most consistently used category across analysts and alert types, whereas some of the other categories vary more depending on the alert being investigated [20]. This suggests that effective triage requires access to multiple kinds of information, but that not all evidence contributes equally in every case.

The Kersten categorization provides a structured vocabulary for describing the information sources analysts use during alert triage. The categories distinguish between information about the alert itself, nearby or previous alerts, contextual host evidence, and evidence about the alleged attack. Table 2.1 summarizes the Kersten-derived factors, while Table 2.2 summarizes the broader information categories used by SOC analysts when classifying alerts.

Table 2.1: Kersten-derived factors relevant to alert triage.

Factor	**Meaning**
SiS	Signature specificity
SiA	Signature age
AH	Alert history
SA	Surrounding alerts
RL	Related logs
TSI	Traffic stream information
THB	Target host behaviour
AI	Attacker information

Table 2.2: Categories of information SOC analysts employ to classify alerts

Information Category	**Definition**	**References**
Relevance indicators	Information to classify whether the alert under investigation is even relevant for the SOC, based on the signature and the scope of the customer.	[1, 21]
Additional alerts	Alerts related to the current alert that the analyst is investigating. This may be previous instances of the same alert triggering or alerts that surround the current alert.	[21, 20]
Contextual information	Information about the behavior and other observables of the involved internal host.	[1, 21, 20]
Attack evidence	Any evidence relating to the alleged attack including the type of attack, attacker and any indication of success.	[1, 21]

2.4 Transformers and Attention

In 2017, Vaswani et al. [42] introduced transformers, which are now a central foundation for modern language models [27]. In a transformer, text is represented as a sequence of tokens where each token is converted into a numerical representation. Positional information is then added so that the model can use the order of tokens.

The central part of the transformer is self-attention. Self-attention lets each token use information from other tokens in the same input. This enables the model to relate words, commands, host names, and other types of information to each other when giving an answer. Additionally, transformers use multi-head attention, which means several attention mechanisms are applied in parallel. This allows the model to represent different relationships in the same input [42].

This is relevant for SOC alert analysis because the input to an LLM can contain more than the alert text alone. It can also contain related logs, host information, report instructions, and other contextual evidence. The model can therefore generate an answer based on several pieces of information in the same context. However, more context is not always better. Irrelevant information can distract the model and reduce performance, even when the model can solve the same task without that extra information [36]. This limitation is important in SOC settings, where useful evidence often appears together with large amounts of noisy background data.

2.5 Large Language Models

In machine learning, a language model is a neural network trained on a corpus for natural language tasks in order to learn patterns and generate text. A large language model (LLM) is a type of pre-trained language model that consists of 10B+ parameters [46]. The defining characteristic of an LLM is not only its size, but also its ability to generalize across many language tasks without requiring specialization. LLMs usually refer to models that use the Transformer architecture.

Large language models (LLMs) are transformer-based language models trained on large amounts of text, which enables them to understand prompts, identify patterns in language, and generate human-like text. In practice, this gives them a broad set of capabilities, such as summarization, extraction, classification, question answering, and the generation of structured natural-language reports. These capabilities have made LLMs increasingly relevant in cybersecurity, where analysts often need to interpret heterogeneous textual and semi-structured information such as logs, alerts, phishing messages, and incident reports [16, 19].

In a SOC context, this is particularly relevant because alert analysis is not only about assigning a label, but also about combining different pieces of context and presenting a reasoned explanation. Prior work has shown that LLMs can be useful for tasks related to log analysis and security reporting, but it has also shown that they are prone to hallucinations, trust issues, and sensitivity to the surrounding context [16, 31]. These limitations are important when LLMs are considered as support tools for contextualized alert triage in SOC environments.

2.6 Chain-of-Thought Prompting

Chain-of-thought (CoT) prompting is a method for improving LLM performance by encouraging the model to generate intermediate reasoning steps before producing a final answer [44]. Wei et al. showed that prompting a model to explicitly work through such intermediate

steps can improve performance on several reasoning tasks, including arithmetic, commonsense, and symbolic reasoning [44]. This line of work later contributed to the emergence of reasoning-oriented models such as OpenAI's o1 and DeepSeek-R1 [3]. The general idea is that better performance can be obtained when the model is guided to reason through the problem in multiple steps rather than responding immediately. However, these newer reasoning models are not simply ordinary LLMs prompted to show their reasoning. Instead, they are specifically trained to perform multi-step reasoning more effectively. OpenAI describes o1 as being trained with reinforcement learning to use its chain of thought more productively, while DeepSeek-R1 uses a multistage pipeline combining supervised fine-tuning and reinforcement learning to improve reasoning behaviour [15].

2.7 LLM Agents and Tool Use

When an LLM is prompted, it is limited to the information present in that prompt and the model's learned parameters. By enabling tool use, the LLM-based agent is able to request external information or functionality during execution. This is applicable when tasks require external information or functionality, such as additional logs, alert history, or host data. Previous work shows that tool selection and tool use are important capabilities, but that they can also be problematic when a large set of tools or similar tool descriptions are available [25, 37, 32].

2.8 Model Context Protocol

Model Context Protocol (MCP) is a standardized protocol for exposing external systems and resources to an LLM-based agent. In practice, this allows the agent to access external functionality through a defined set of tools that it can invoke during execution [18]. A tool normally has a name, a description, an input schema, and a returned output. The name and description tell the model what the tool is for, the input schema limits which arguments can be sent, and the output becomes additional context for the model's next step.

In SOC alert analysis, MCP can turn evidence sources into explicit information channels. Instead of placing all available logs directly in the prompt, an agent can request a specific type of context, such as related logs, alert history, surrounding alerts, or target-host behaviour. This also makes the tool description important: it does not only describe a technical function, but also tells the model how the evidence should be interpreted. For example, an alert-history tool can state that previous alerts are context for the focal alert, not proof that the current alert is malicious.

2.9 Evaluation of SOC Analysis Reports

For SOC analysis reports, output quality depends on more than just the final verdict. A report should be complete enough for the analyst to follow, grounded in evidence, and structured in a way that makes the reasoning easy to understand. This is important since analysts rely on contextual information when deciding whether an alert is meaningful [21, 20].

The LLM-as-a-judge methodology leverages large language models to autonomously evaluate the outputs of other system configurations [30]. By prompting an LLM to serve as a proxy for a human evaluator, it becomes possible to automate the testing workflow and bypass the otherwise required manual human-in-the-loop assessments. This automation accelerates the testing pipeline, allowing the evaluation of larger datasets than would otherwise be feasible.

Evaluating an AI analyst's operational capabilities requires a high volume of data to reliably map strengths and weaknesses. While the manual review of extensive log data is vulnerable

to human error driven by analyst fatigue [39], a properly prompted LLM provides consistent data processing capacity at scale.

However, from an architectural and risk management perspective, it is critical to state that this method does not eradicate error. Rather, it trades the unpredictable nature of human cognitive exhaustion for the systematic, measurable biases of a language model. Acknowledging and controlling for these inherent model biases is therefore a strict requirement for a valid evaluation design [7].

MESSALA is designed to provide automated feedback and quantitative scores that closely resemble human expert judgment [30]. It represents previous work on evaluating SOC-style reports.

In MESSALA, the score should be understood as a report-quality score rather than a verdict-correctness score. A low score means that the report weakly satisfies the SOC-report criteria, for example by missing evidence, clear reasoning, or actionable conclusions. A high score means that the report strongly satisfies the criteria, with clear conclusions, relevant evidence, and useful guidance for analysts. The framework uses a practitioner-derived checklist, report-specific granularized checks, and a multi-perspective evaluation step that combines high-level and in-depth judgements [30].

2.10 ELK Stack

To efficiently manage the vast amount of data produced when monitoring network and system traffic, suitable tools and methods are required. One widely used option is the ELK stack. Three different open-source tools are combined to create the ELK stack, namely Elasticsearch, Logstash, and Kibana [38].

Elasticsearch is an open-source search engine, and the latest versions (8.16 and onward) are released under the three open-source licenses AGPL v3, SSPL 1.0, and Elastic License 2.0 [11]. One of the most important features of Elasticsearch, from a security analyst's point of view, is its ability to provide real-time search and analysis capabilities [8].

Logstash is an open-source data collection engine that can dynamically combine different datasets and normalize the content to the desired destination [13].

Kibana is an open-source interface used to analyze, visualize, and manage the data in Elasticsearch [12].

3 Related Work

This chapter presents prior research related to SOC analysis, log analysis, and AI-assisted security operations.

3.1 LLMs for Security Log Analysis

To address the explainability issues of Predictive AI, research has been directed towards LLMs. A systematic review by Hasanov et al. [16] analyzed 177 articles and identified that LLMs are now a primary tool for defensive cybersecurity. Further, it was shown that LLMs have demonstrated significant proficiency in analyzing data sources such as system logs and phishing emails. Unlike traditional parsers that rely on rigid templates, recent studies [19] show that transformer-based models can identify anomalous events in heterogeneous log data without handcrafted preprocessing.

However, a gap remains in how these models are applied in an operational SOC environment. Karlsen et al. [19] focused primarily on "isolated log-line classification," treating events as independent sequences without modeling the broader relationships between alerts. Furthermore, Hasanov et al. [16] highlight a crucial limitation in relation to operational trust, which is critical in an SOC environment. They found that LLMs are prone to hallucinations, which in turn leads to inaccurate outputs. This suggests that even though LLMs are an effective tool for the parsing of logs [16], research is still required to determine how to integrate them into triage workflows to reliably reduce cognitive load.

3.2 AI Support for SOC Analysts

To further increase the capabilities of a SOC environment, Baruwal Chhetri et al. [5] looked at how humans and AI could be companions in a SOC environment. The researchers propose the A^2C framework for defining three modes of decision making. The automated mode where alerts that match well-understood patterns are routed to an automated validation queue where the AI autonomously classifies them as false positives or genuine threats. The expert mode is where skills of the AI cannot be trusted, thus the alert is instead routed to the human analyst, as a human-in-the-loop mechanism. Here, the AI may either defer passively,

providing no recommendations to avoid biasing the analyst, or provide augmented deferral with contextual information such as historical data on the flagged IP, device information, or external threat intelligence. For novel or ambiguous threats, the framework envisions collaborative exploration, where the analyst and AI engage in iterative dialogue to jointly investigate the alert.

The paper presents the A^2C framework, a conceptual framework for human-AI collaboration in security operations centers (SOCs). The framework is based on the idea that the AI and the human analyst should have shared situation awareness. The framework identifies three components of shared situation awareness: self-awareness, teammate awareness, and world awareness. Self-awareness is about each agent (human and AI) knowing their own limitations and capabilities. Teammate awareness is about each agent understanding the other agent's expertise level, cognitive load, and preferences. World awareness is about having a shared model of current goals, the status of tasks, and the overall security environment. The paper discusses potential ways to implement the framework, but it remains conceptual. The authors suggest that imitation learning and inverse reinforcement learning could be used to learn analyst investigation patterns, and reinforcement learning with human feedback could be used to improve AI recommendations. The work does not include a practical implementation, which leaves the operational effects of AI-human collaboration in SOC environments unresolved.

Relatedly, work on explainable AI in intrusion detection [31] shows that just knowing what event is malicious or not is not sufficient. Analysts need to know more than just what event was malicious. Rather, an explanation of what features and reasons contributed to the AI's classification is needed for analyst-facing decision support.

3.3 LLM Agents in Cybersecurity

While log analysis provides the foundation for detection, recent literature has expanded toward the development of autonomous AI agents capable of executing complex security tasks. Hasanov et al. [16] highlight that systems integrating LLMs with SIEMs can significantly improve operational transparency by explaining why a specific action was taken, rather than only executing black-box rules.

To validate the effectiveness of these agents, NetSecGame, a simulated network security environment was implemented [34]. The study compared LLM-driven agents with traditional reinforcement learning agents. The result showed a significant performance difference between human testers, reinforcement learning agents, and LLMs. LLMs showed better performance when faced with planning and multistep cybersecurity tasks.

There is a gap remaining regarding the application of these agents within a Tier 1 triage workflow. While Rigaki et al. [34] focused on "winning" abstract defense scenarios, there is limited research evaluating how such agents perform the specific, nuanced task of alert triage within a realistic network architecture. Compared with abstract game environments, realistic network settings are more suitable for assessing practical utility in triage time reduction and threat contextualization using simulated enterprise data.

4 Method

This chapter presents the methodological choices used to design, implement, and evaluate the proposed alert analysis pipeline. It explains how the study combines controlled data generation and comparative evaluation in order to answer the thesis' research questions. Further, the chapter describes the lab environment, the generation of benign activity, the operationalization of contextual factors, and the design of the evaluation process. This chapter motivates the tool and framework choices made in the implementation, defines the final system variants compared under controlled conditions, and explains how diagnostic development findings were used to freeze the selected K-MCP configuration before the final evaluation.

4.1 Research Design and RQ Mapping

This section aims to address how the different research questions were answered throughout the thesis process.

4.1.1 How do contextual tool access and prompt-level design choices affect the performance of an LLM-assisted SOC alert-analysis pipeline compared with a base LLM agent?

To answer the first research question, this thesis investigates whether the pipeline can be structured to perform the main steps of SOC alert analysis in a useful manner. Rather than treating the model as a single response system, the pipeline is designed as a workflow in which the LLM receives an alert, reasons over relevant context, and produces a structured analysis report. This makes it possible to study not only whether the pipeline reaches a reasonable verdict, but also whether it can support the analytical process in a way that resembles practical SOC triage. The question is therefore addressed through the design and evaluation of a prototype pipeline, together with a comparison of different system variants built for this purpose.

4.1.2 How do different *Kersten* factors contribute to the performance of the LLM pipeline?

A collection of research surrounding what information is important for a human security analyst was collected, heavily based on the papers from Kersten et al. [20, 21]. These factors were later used individually by a specific tool in an MCP server that was created (this is further explained in 4.8). By being allowed to toggle the usage of a specific Kersten factor an ablation study could be conducted. By performing an ablation study each factor's value to the final model is exposed and can be assessed individually. This helps with identifying relevant knowledge for an agent performing alert triage.

4.1.3 How can a controlled benign dataset be created in a SOC context?

To answer the third research question, this thesis does not rely only on existing public datasets. Existing datasets often lack either relevant background noise or the type of benign alerts that the agent is intended to assess. Therefore, the benign part of the dataset was generated in a simulated SOC environment using Windows virtual machines, Elastic-based data collection, and GHOSTS-generated user activity. This made it possible to create benign alerts that still required contextual interpretation, which is important when assessing alerts for human security analysts [21]. The malicious alerts were not generated in this lab environment. They were taken from the public MORDOR 2019 dataset and added to provide malicious examples for binary scoring.

4.2 Lab Environment

In order to create data that can be used as noise for the investigating analyst, an environment was constructed using three workstations on a Local Area Network (LAN). In a simulation environment where security-relevant benign activity is generated and monitored, several requirements must be handled to keep the simulation controlled and safe. Primarily, it was ensured that the environment existed on a LAN that was controlled and isolated from other production networks present at the site. The LAN consists of three virtual machines all running Windows 11 Pro. The purpose was to simulate an office enterprise environment, where Windows hosts were chosen due to Windows prevalence in such contexts [2]. The three virtual machines each had a GHOSTS client installed to generate benign traffic, and an Elastic agent that captured and monitored the virtual machines.

As seen in Figure 4.1, each virtual machine, along with the physical hosts, operates on the same LAN. *HOST A* hosts VMware and the installed virtual machines, whereas *HOST B* hosts the GHOSTS API and the complete Elastic stack (Elasticsearch, Logstash, and Kibana) with the addition of the Fleet server. Since both the ELK stack and the Fleet server are installed on the same PC (*HOST B*), the internal communication allowed for a simple setup. Through the switch in the lab setup, internet access is provided. This was needed to make activity from the virtual machines more realistic. For example, making a virtual machine browse the internet and search for websites can be a practical way to simulate ordinary user activity. Since it is the virtual machines that generate the benign traffic used in the dataset, *HOST A* provides a bridged network to the virtual machines. This allows internet access through *HOST A*'s physical network connection and allows each virtual machine to get its own IP address on the LAN.

4.3 Data Generation

The data collection provides the foundation for the later evaluation by generating benign alerts together with surrounding background logs. Because the integration was controlled,

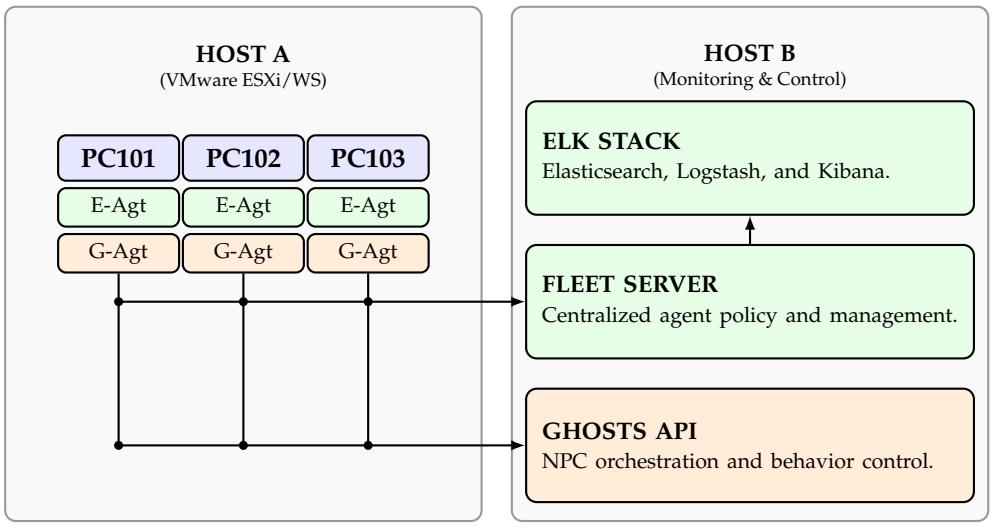


Figure 4.1: Testbed architecture detailing the monitoring and orchestration routing between Host A and Host B.

the resulting benign alerts could be traced back to specific host actions. This supports reproducibility and makes it possible to compare evaluated variants on the same material. The malicious alerts were added separately from the public MORDOR 2019 dataset and were not generated by this lab setup.

Creating the lab environment from scratch allowed for a highly controlled and flexible method for generating data. To generate the data, the GHOSTS framework was implemented as described below. GHOSTS made it possible to generate traffic that comes close to what an active network environment would look like and, by extension, allows for an enhanced simulation environment [9].

4.3.1 GHOSTS

The GHOSTS framework was used as the lab's benign-activity generator. GHOSTS is designed to generate high-fidelity network traffic for cyber simulations and exercises, which made it suitable for creating repeatable user-like activity in the controlled SOC testbed [41]. In this thesis, GHOSTS was used only to generate benign host activity and background noise. Malicious alerts were instead added separately from the public MORDOR 2019 dataset, as described in Section 4.6.

By installing GHOSTS clients on each virtual machine, the study could generate benign user activity on the hosts by using timelines. These timelines were created based on different administrative and office activities that normally occur in enterprise networks. To keep the simulation role-consistent, administrative activity mainly occurred on *IT-WS-01*, the virtual machine with an administrative account as its active user. This was important, as a normal HR user, for example, should not be scheduling tasks and creating accounts.

4.3.2 GHOSTS Timeline

Each host in the simulation environment was assigned a baseline `timeline.json` file that defined recurring benign activity. This allowed the hosts to maintain a stable activity profile throughout the data-generation period while still reflecting differences in user role and workstation function.

To supplement the baseline timelines, timeline partials were used for selected hosts. A timeline partial, presented in Table 4.1, is a temporary JSON-defined task sequence pushed through the GHOSTS API to a specific host. Once executed, the agent returns to its original baseline timeline. This made it possible to inject controlled short-lived activities without permanently modifying the host's default behavior or introducing extra artefacts from manually starting and stopping services. The full JSON definitions of the timeline partials are provided in Appendix A.

IT-WS-01 on PC103

IT-WS-01 on PC103 represented the administrative workstation in the simulated environment. For this host, timeline partials were used to introduce short administrative actions that complemented the baseline activity profile. These partials were designed to reflect tasks that are plausible for an administrative user in an enterprise setting, such as configuration checks, scheduled-task creation, service changes, and log export.

One example is `admin_schtask.json`, which created a scheduled task on the host. This is consistent with Barrett et al. [4], who describe the use of custom tools and rehearsed procedures as part of normal system administration practice in enterprise environments. In the context of this simulation, scheduled tasks served as a practical mechanism for representing recurring maintenance and configuration-related activity.

Table 4.1: GHOSTS Administrative Timeline Partials and Literature Grounding

Base Action	Grounding
`defender_check`	Admin work is information-intensive; checking security status maintains essential situation awareness [4].
`firewall_change`	Configuration and maintenance are core sysadmin tasks in enterprise environments [4].
`powershell_inventory`	PowerShell is a legitimate management tool utilized to collect system state and telemetry [28].
`service_change`	Continuous infrastructure maintenance dictates that service state modification is a standard operational requirement [4].
`schtask`	Scheduled tasks serve as the functional representation of rehearsed procedures and automated maintenance scripts [4].
`temp_user`	Active Directory acts as a centralized administration service where account management is a primary function [4].
`log_export`	Log collection provides the trustworthy system information necessary for operational verification and auditing [45].

HR-WS-01 on PC102 & HR-WS-02 on PC101

Both PC101 and PC102 acted as benign hosts and the main purpose of these machines was to generate background noise that the LLM pipeline would have to navigate through in order to find the useful information. Table 4.2 summarizes the office timeline partial used for these HR and office workstations.

Table 4.2: GHOSTS HR and Office Timeline Partials and Literature Grounding

Base Action	Grounding
`office_pc101_permanent` `office_pc102_permanent`	Represents standard behavior and the utilization of normal office applications for non-technical staff [23].

4.4 Data Collection and Storage

To store and collect data, the aforementioned SIEM platform, the ELK stack presented in Section 2.10, was chosen. Additionally, the ELK stack offered good performance compared to enterprise competitors [38], which further strengthened its relevance for this project.

4.4.1 Elastic Defend

Since the ELK stack was used to monitor and detect anomalies, a suitable detection logic was to enable Elastic Defend on each of the virtual machines' Elastic agents. By doing this, the Elastic agent automatically collected a set of data from the hosts, decreasing the time overhead for this thesis. Elastic Defend enabled functionalities like detection, prevention, response and deep visibility on the monitored host [10]. However, in this thesis, the primary functions were detection and deep visibility, meaning that prevention and response were disabled.

4.4.2 Windows Auditing Policies

Windows lets one choose what to monitor and what to audit on a host machine. This functionality enables more verbose and deeper auditing capabilities. The main reason to increase the volume of data is to amplify the earlier-mentioned *needle in a haystack* problem. However, it is strongly recommended that a computer, especially within an organization, should have additional auditing policies enabled [45].

The audit policies enabled for each of the virtual machines are as follows:

- **Audit logon** - Enabled for success and failure - this tracks authentication attempts.
- **Audit file system** - Enabled for success and failure - allows for monitoring of access to files and directories.
- **Audit registry** - Enabled for success and failure - this tracks modification to the Windows registry.
- **Audit process creation** - Enabled for success and failure - this creates a log entry every time a process is started or stopped.
- **Audit filtering platform connection** - Enabled for success and failure - this captures kernel-level data regarding network connections allowed or blocked.

4.4.3 Automatic Data Collection Run

The benign logs and alerts that were used in the final dataset were generated from a 5-day long automatic data collection run. During this period, the system was left untouched by external sources such as the authors in order to affect the outcome as little as possible. The GHOSTS scripts handled the execution of events and the Elastic agents collected telemetry from each host regularly.

4.5 Benign Alert Retrieval and Sampling

Over the course of a 5-day simulation period 149 alerts were generated from the 3 different virtual machines. However, due to this project's budgetary constraints the total amount of alerts, 149, exceeded the expected amount for benign alerts. To counter this a Python script that retrieves a subset of the alerts based on a sampling technique was created.

Sampling The sampling method used for the alert retrieval was structured in the following manner:

- **Deduplication:** To avoid redundancy from near-identical alerts a light deduplication step was applied prior to sampling. Any alert that belongs to a *rule.name* + *host.name* + *5-minute* bucket the script selected a single alert to keep in the final alert dataset.
- **Proportional stratified random sampling:** After deduplication the remaining alerts were grouped by *rule.name*, where each rule constituted one stratum. A proportional stratified random sample was then drawn, meaning that each stratum contributed alerts to the final dataset in proportion to its size in the deduplicated sampling frame. Random selection within each stratum was performed using a fixed random seed (42) to ensure reproducibility.
- **Sample size calculation:** The target sample size was determined using a finite population sample size formula with a confidence of 95%, a margin of error of 10% and a conservative assumption $P = 0.5$. Concretely, this means that an estimated proportion from the sampled benign alerts would be expected to fall within 10 percentage points of the corresponding proportion in the full alert population in 95 out of 100 repeated samples. A 10% margin of error was accepted because the sample was intended to support controlled model evaluation under project budget constraints, rather than to estimate production SOC alert rates with high precision. For an initial alert population of 149, this resulted in a final target sample of 59 benign alerts.

This is done to ensure that the alert dataset is not dominated by the noisiest rule families and by extension create a more varied dataset to evaluate the configurations' performance.

4.6 Malicious Alert Reference Set

The final evaluation set also included 17 malicious alerts from the public MORDOR 2019 dataset. These alerts were not generated in the lab environment used in this thesis. They were added so that the evaluation would include a malicious class and so that a configuration could not get a misleading result by classifying every alert as benign.

The main dataset contribution of this thesis is still the controlled benign alert dataset with surrounding background noise. The malicious alerts are included to make the evaluation set contain both benign and malicious examples.

4.7 Prototype Implementation

This section describes the prototype implementation used in the evaluation. The focus is on how the LLM-based alert analysis pipeline was structured.

4.7.1 Language Model Selection

This thesis uses GPT-5.4 as the language model in the evaluated alert analysis pipeline. The model is a fixed component in the study, since the goal is to compare pipeline configurations instead of comparing different language models. This makes it easier to interpret performance differences as effects of the surrounding system design, contextual information, and evaluation setup. Additionally, this follows the general principle that language models should be evaluated under standardized conditions when comparing behaviour across tasks and metrics [26].

A limitation of this choice is that GPT-5.4 is a proprietary, provider-hosted model. The full training data, internal architecture, and inference behaviour are not publicly available [33]. Therefore, the results should be interpreted as specific to GPT-5.4 and to the experimental setup used in this thesis.

4.7.2 The LangChain Ecosystem

The LangChain ecosystem provides a family of frameworks and platforms for building, observing, and evaluating LLM applications. It provides a wide range of abstractions for prompting, model interaction, tool use, and orchestration [24]. Specifically, LangChain is the foundational framework for building LLM-based applications within the LangChain ecosystem, where the two other important components used in this thesis are LangGraph and LangSmith. LangGraph extends LangChain by providing more control, features like internal logic, stateful agents, amongst others. LangSmith provides a platform for observing and evaluating LLM-based applications. The LangChain ecosystem provides a seamless symbiosis between working in code and in the web-based GUI.

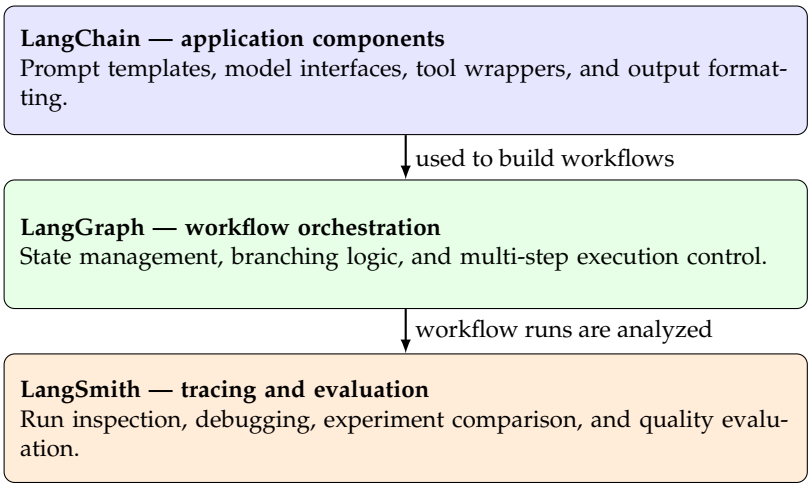


Figure 4.2: Role separation in the LangChain ecosystem for this study. LangChain provides reusable components, LangGraph orchestrates execution, and LangSmith supports tracing and evaluation.

LangChain was chosen because of its rich abstraction of components relevant to LLM-based applications in this thesis (e.g., orchestration logic, interfaces, tracing, and evaluation), as well as because it is model-agnostic and free to use. Being able to control the internal logic through LangGraph enabled the construction of simple configurations and a modular path to extending it to more advanced configurations, while keeping the same foundational architecture and allowing for easy evaluation.

Why LangChain was chosen

It provided a model-agnostic streamlined abstraction for prototyping and implementing LLM workflows. It allowed both modification of fine details through LangGraph and use of standard primitives for simple implementations through LangChain. It also offered seamless integration with LangSmith, which allowed observability through tracing, debugging and evaluation.

LangGraph as the workflow layer

LangGraph was used as the layer for building the agents. LangGraph provided a stateful and explicit way to program agents and their variants. The two fundamental ways to design the agent were by manipulating the graph's nodes and edges. Nodes were the functionality or actions that the graph ran, such as LLM runs or functions.

To start off, a shared LangGraph shape presented in Figure 4.2 was used. The agents took the alert JSON as input, and after processing the alert, the agent output the full alert report.

The base agent consisted of a single node that took the alert JSON as input. The Kersten-MCP agent (K-MCP agent) referred to the agent variant that operationalized Kersten-derived factors of analysis through MCP tools.

LangGraph was used as the workflow layer for the prototype agents. Each alert was processed as an independent triage task with the same outer graph shape: the alert was loaded into a shared `SocAgentState`, passed through one analysis node, and then returned as a structured triage result. This kept the orchestration stable across variants while allowing the internal implementation of the analysis node to differ.

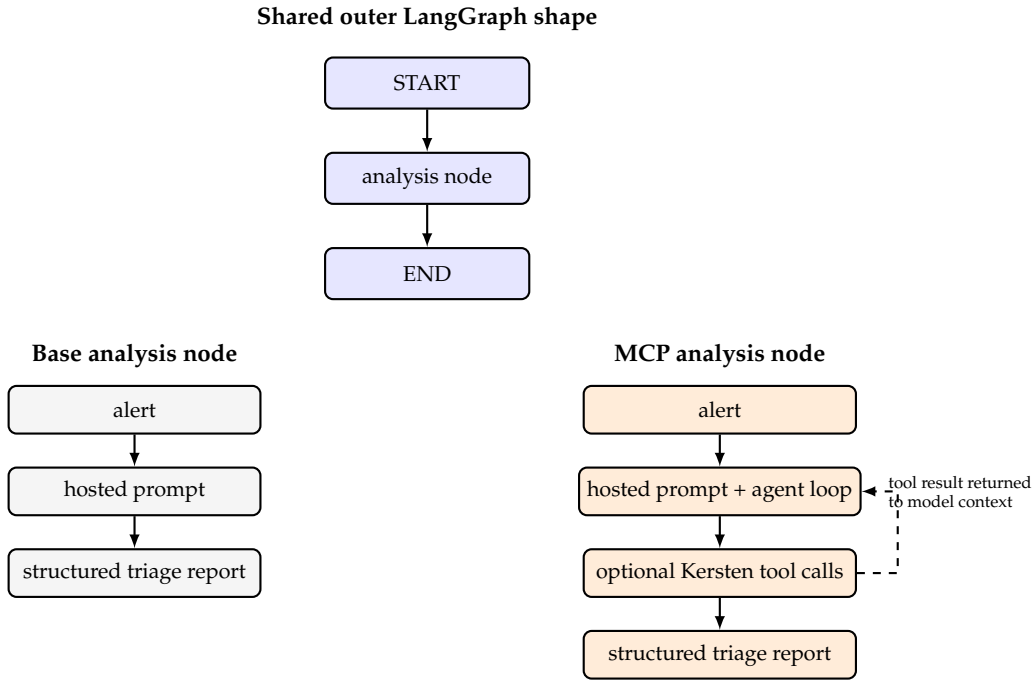
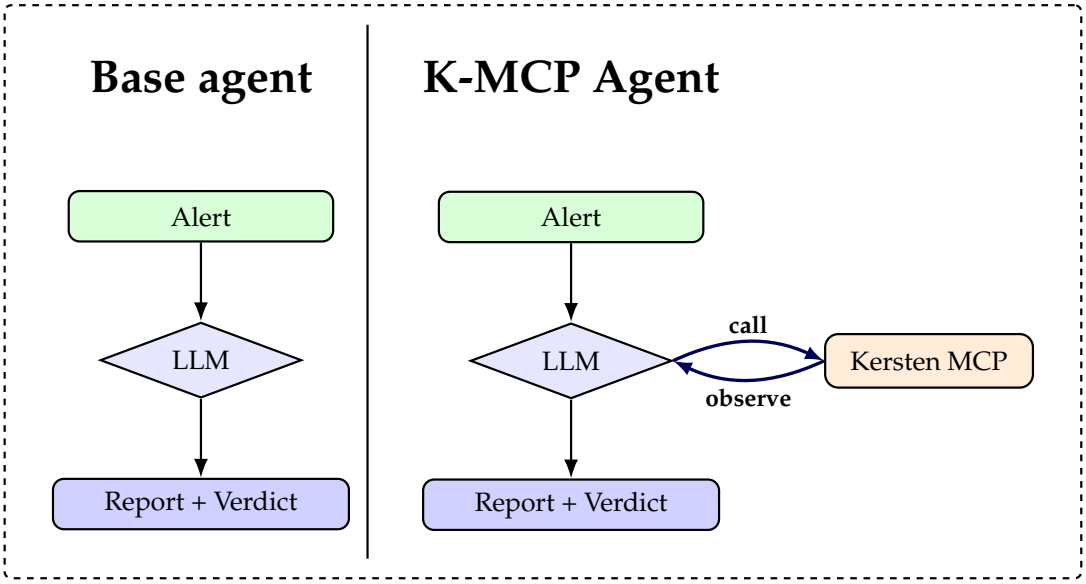


Figure 4.3: Shared LangGraph structure and differing analysis-node internals for the base agent and K-MCP agent.

The base agent used a single LangGraph node, `run_base_analysis`, which rendered the alert into the hosted triage prompt, invoked the model, and returned the final state update.

The K-MCP agent used the same outer structure, but replaced the internal analysis step with `run_langchain_create_agent_mcp`. This node loaded the enabled Kersten-factor tools, passed them to the LangChain `create_agent` loop, and allowed the model to request tool outputs before producing its final report. Tool outputs were therefore not treated as separate artifacts; they were incorporated into the model context and reflected in the generated analysis.



This structure, presented in Figure 4.3, supported the experimental design because the same state representation and output contract could be reused across the base agent and K-MCP agent. Internally, the structured output was represented as `TriageOutput`, containing `analysis: str` and `malicious: bool`. At runtime this was mapped to the evaluation-facing output, where `response` contained the generated analysis report, `malicious` contained the binary verdict, and `mcp_rounds` was present for MCP runs to record tool-use rounds. Since the enabled tool subset was controlled by the selected Kersten factors, the same graph could be reused for ablation experiments while changing which contextual factors were available to the agent.

4.8 Operationalizing Kersten Factors Through MCP

This section explains how the Kersten factors introduced in the background were translated into MCP tools for the K-MCP configuration to use. The two main purposes were to make relevant context available during alert triage and to ensure experimental control so that the individual factors could be enabled, disabled and compared.

4.8.1 Relevance of Factors

There were many possible ways to assist an LLM. Giving an LLM visual capabilities could be useful in some cases, but most likely not in a SOC environment. Thus, it was important that the tools considered for experimentation were actually relevant within a SOC environment, and specifically within the investigation and analysis process, since that is what the analysis agent was for. Thus it was of interest to know which relevant factors influenced a SOC analyst in their process. A literature review was conducted on the subject. The goal was to find a list

of factors of analysis that real SOC analysts consider. Kersten et al. identified a list of these essential factors of analysis in their paper by conducting interviews with ten Tier-1 analysts employed within a SOC [21]. This paper provided a list of factors of analysis that were then used as a guideline to assess whether a proposed LLM tool has relevance in the SOC analysis context.

4.8.2 Rationale for Factor-Level Toggling

A SOC analyst is faced with alerts and events that are information-dense and tacit [21]. By turning this information into controllable units it is possible to evaluate what factors of analysis that are important to this thesis' dataset. It is important to know what factors contribute to a successful triage in order to clearly identify what information that is of essence for an agent analyst. A way of detecting these factors of analysis is by conducting an ablation study. Ablation studies allow for assessing each factor's contribution to the base model and by extension identify what the relevant factors are for alert assessment [35]. If all are available together the model may perform better but the source of that improvement becomes opaque. Further, Kersten et al. mentioned that lack of structure in tier-1 analyses makes inefficiencies in the analysis hard to spot [21].

4.8.3 MCP Tool Surface

To operationalize the factors of analysis, the external capability layer was implemented through the Model Context Protocol (MCP). MCP was suitable for this purpose because it exposes tools as explicitly named interfaces with schema-defined inputs and structured outputs, which makes each factor easier to inspect, constrain, and evaluate in isolation [40]. This was important for the present thesis, since the goal was not only to give the agent more context, but also to preserve experimental control over which factor was made available at a given time.

The MCP surface was therefore kept deliberately small. Instead of creating several near-identical tools for the same Kersten factor, each factor was represented by one canonical tool, while the amount of returned evidence could be controlled through parameters rather than through additional overlapping tools. This choice was motivated both by protocol design and by prior tool-use research. API-Bank shows that planning, retrieving, and calling tools remain challenging even in comparatively moderate tool settings [25], while ToolLLM and ToolBench further demonstrate that tool-use performance depends strongly on how well models can select and sequence APIs from large candidate sets [32]. More recently, TOOLRET shows that retrieval quality drops when agents must choose from large collections of similar tools, and that this reduction propagates to lower end-to-end task performance [37]. For this reason, a smaller and more semantically distinct MCP tool surface was preferred, since it supports clearer factor-level ablations while reducing unnecessary ambiguity in tool selection.

4.8.4 Progressive Disclosure and Tool-Use Guidance

The MCP tools were designed around progressive disclosure. Rather than returning all available context immediately, each factor first exposed a compact summary that could be inspected quickly. More detailed evidence could then be retrieved when the summary indicated that a factor was relevant to the focal alert. This was intended to reduce context overflow and inefficient retrieval, while still allowing the agent to drill down into the evidence needed for a supported verdict.

Initial MCP diagnostics also showed that tool availability did not guarantee useful tool use. Related logs (RL) sometimes contained the most concrete grounding evidence, but the agent did not reliably inspect or use that evidence when it was available. The prompt and tool

guidance were therefore adjusted so that RL was treated as a primary grounding source for checking whether the alert was supported by concrete events. This design choice reflects a broader methodological lesson from the development process: the Kersten factors should not only be exposed as tools, but also described with enough role-specific guidance for the agent to know when and how each factor should influence the analysis.

4.9 Development Iterations and Final Configuration

The development process was used to diagnose how the agent used contextual evidence before the final evaluation variants were selected. These observations should be read as design diagnostics rather than as the final result comparison. The development history showed that adding more context was useful only up to a point: the agent needed analyst-relevant evidence to reduce false positives, but unrestricted contextual signals could also make the agent over-attribute suspicion to the focal alert. The final reported classification result therefore uses clean complete same-configuration reruns of the selected scoped K-MCP configuration rather than the best single historical run.

4.9.1 Iteration 1: Direct Baseline and Initial K-MCP Comparison

The first development step compared the base SOC agent with an initial K-MCP-enhanced agent. The purpose was to test whether exposing Kersten-derived MCP tools changed the agent's analysis compared with reasoning only from the alert input. This comparison showed that MCP access changed the agent's reasoning, but it also revealed that tool availability alone was not enough. The agent needed guidance about when a contextual factor should be used and how strongly it should influence the final verdict.

4.9.2 Iteration 2: Related Logs Guidance

Early MCP runs showed that the agent did not reliably use Related logs (RL), even when RL contained important grounding evidence. Since RL can contain concrete host, protocol, connection, DNS, and alert-log evidence, this was a problem for both verdict quality and report grounding. The prompt and tool guidance were therefore adjusted so that the agent was encouraged to inspect and use RL when it was available. This iteration established a broader design lesson: exposing MCP tools is not the same as making the agent use those tools well.

4.9.3 Iteration 3: Progressive Disclosure

The next issue concerned context volume and retrieval efficiency. Rich MCP context could overwhelm the agent or lead it to retrieve detailed evidence before it knew whether that evidence was relevant. The K-MCP implementation was therefore organized around progressive disclosure. Each factor first exposed a compact summary, and the agent could then request more detailed evidence when the summary appeared relevant to the focal alert. This preserved access to rich context while reducing the risk that irrelevant or excessive evidence would dominate the analysis.

4.9.4 Iteration 4: Global AH/SA Contextual-Evidence Nudge

Qualitative inspection of generated reports revealed a more specific failure mode for Alert History (AH) and Surrounding Alerts (SA). These factors were useful for understanding whether an alert was isolated, recurring, or part of a nearby activity pattern, but the agent sometimes used them in a paranoid or guilt-by-association manner. In those cases, nearby or historical alerts were treated as proof that the focal alert was malicious, even though those

surrounding alerts could themselves be benign or false positives. Figure 4.4 shows an example of this contextual overreach.

Excerpt from a diagnostic K-MCP response:

"Additional later alerts on the same host show suspicious `schtasks.exe` *task creation by user* `IT-WS-01` *with similar host-profiling/output-redirection behavior, which increases concern that the focal event is part of a broader unauthorized discovery/persistence chain rather than a one-off benign admin action."*

Figure 4.4: Example of contextual overreach in a diagnostic K-MCP report. The agent used surrounding or historical alerts as incriminating evidence against the focal alert, even though such alerts are only contextual signals unless they provide direct support for the focal event.

4.9.5 Iteration 5: Scoped K-MCP without AH/SA

Repeated evaluation showed that prompt-level and tool-local warnings did not make full AH/SA context reliable enough to use as the headline final configuration. Although AH and SA are meaningful concepts in human triage, this implementation could still treat them as stronger evidence than warranted for the focal alert. The selected configuration therefore removed AH and SA while retaining Signature specificity (SiS), Signature age (SiA), Related logs (RL), Traffic stream information (TSI), Target host behaviour (THB), and Attacker information (AI). This configuration is referred to as the selected scoped K-MCP configuration.

4.9.6 Historical Development Runs

Historical runs were used to explore prompt, tool, and context-design choices. They show why the selected scoped K-MCP configuration was chosen.

4.9.7 Final Selected Configuration

The selected scoped K-MCP configuration used in the final RQ1 comparison combines Related logs guidance, progressive disclosure, and removal of AH/SA context. The final classification result is reported as the mean over clean complete same-configuration reruns. The development process therefore supports a narrower design lesson than simply adding more context: LLM-assisted SOC triage benefits from contextual evidence when that evidence is carefully scoped to the focal alert.

4.10 System Variants

The final evaluation used two comparisons on the same mixed alert set. The first comparison addressed RQ1 by comparing the base SOC agent with the selected scoped K-MCP configuration. The second comparison addressed RQ2 by summarizing clean majority-configuration repeated runs across the evaluated context variants.

Both agents use the same overall architecture: the alert JSON is provided as input, and the agent produces a structured SOC report according to the shared output contract. The difference is that the base agent must reason from the alert input alone, whereas the K-MCP agent is given defined access to selected Kersten-derived factors of analysis through MCP tools. The selected scoped K-MCP configuration uses progressive disclosure, includes Related logs guidance when RL evidence is available, and excludes AH/SA context because repeated evaluation showed that those context types could encourage focal-alert overreach in this implementation.

For the factor-ablation experiment, the evaluated context variants are summarized as repeated condition-level results rather than as a separate derived ranking. Leave-one-factor-out variants are denoted *K-MCP−X*, where X is the removed factor. This ablation design is used to examine how different context configurations affected the agent, while the direct RQ1 comparison evaluates the selected scoped K-MCP configuration against the base agent.

4.10.1 Ablation Studies for Component and Factor Analysis

An ablation study is an evaluation method where a system's configuration is tested by systematically removing its individual parts to assess how important they are for the overall result [35]. By disabling a specific component, such as a certain data input or a reasoning step, and measuring the impact on performance, it is possible to determine exactly what that part contributes to the base model. For example, if removing a component does not negatively affect the final output, it indicates that the specific task is not necessary and can therefore be removed to optimize the system's overhead.

Beyond improving efficiency, this method also serves an academic purpose. When all components are evaluated together, the model might perform well, but the actual source of that improvement remains opaque. By isolating and identifying the functionality of different components, it becomes easier to understand how the system reaches its conclusions. This helps break down the complexity of the model and makes the final configuration much more explainable.

4.10.2 Configuration Freeze

The final variants were selected after the development iterations described in Section 4.9. A configuration was considered ready for final evaluation when the MCP tool surface was stable, each enabled Kersten factor had a defined retrieval role, Related logs were explicitly encouraged when available, and context was scoped to support analysis of the focal alert rather than to act as an independent maliciousness signal. After this freeze, the selected scoped K-MCP configuration was compared directly with the base agent and also served as the headline repeated-run result.

4.10.3 Evaluation Loop

Each evaluated configuration was run on the same alert set and produced one structured SOC report per alert. The alert information was visible to the agent, while the ground-truth label was kept separate until scoring. The detailed notation and scoring workflow are described in Section 4.11.

4.11 Evaluation Design

To properly evaluate the performance of the different configurations, a comprehensive evaluation framework is required. LLMs are notoriously difficult to evaluate due to their nondeterministic nature and the inherent difficulty in evaluating natural language.

To determine the quality of the configurations, a set of metrics is defined. The most important part is the assessment of an alert, which refers to whether the alert is malicious or benign. This is treated as a binary classification problem. Verdict correctness is evaluated with F1 and interpreted together with precision and recall. Accuracy is also reported, but F1 is used as the main verdict metric because it combines precision and recall.

Because repeated executions of the same LLM configuration can still vary, the final classification claims are therefore reported using repeated runs where available. A run was included

in a repeated-run average only if it completed all 76 alerts, had zero task errors, and used the same model, reasoning effort, MCP endpoint, agent variant, and enabled-factor signature.

Meanwhile, the judge, referred to here as MESSALA, is tasked with evaluating different metrics. These metrics are presented by Okada et al. [30]. MESSALA is designed to provide automated feedback and quantitative scores that closely resemble human expert judgment [30]. Further, by utilizing the capabilities of MESSALA, the evaluation can serve as a substitute for a user study, which is a limited resource and would thus complicate the evaluation process.

4.11.1 Important Elements for an Alert Report

The performance of the analysis agent is measured based on the output of its alert report. Three complementary elements of an alert report were defined (see Table 4.3). To cover these elements, a literature review concerning each element was conducted.

Table 4.3: Overview of the evaluation components.

Aspect	Method	Focus
Decision outcome	F1, precision, recall	Verdict correctness and error balance
Report quality	MESSALA	Completeness, structure, support of arguments

Decision outcome focuses on measuring verdict correctness, as the analysis agent's ability to reach the correct conclusion.

Report quality was included because the goal for the analysis agent is not only to provide the correct answer, but also to foster explainability and enable real SOC analysts to understand the rationale behind the verdict. The main purpose of the agent is to provide a SOC analyst with a report of what happened and why the verdict was reached. For the report to be trustworthy and verifiable, the agent is instructed to provide the essential components, list the relevant events, and avoid leaving out information necessary for the analysis.

The analysis agent can have a valid rationale, but MESSALA does not check whether the claims themselves are true, but rather whether the analysis agent points to some evidence for its claims. In order for a generated report to be trustworthy, it is essential that the evidence provided is actually true and not just hallucinated.

4.11.2 Metric per Element

This subsection aims to explain the use of metrics used in this thesis.

Decision-outcome metric

The decision between malicious and benign can be treated as a binary classification problem, and thus F1 score was chosen as the main metric for verdict correctness. F1 is useful because it balances precision and recall within one measure [17]. Precision and recall are also reported separately to make it possible to see which type of error changes between configurations.

MESSALA report-quality metric

MESSALA is an evaluation system based on the work of Okada et al. [30]. MESSALA is used as a metric for report quality rather than verdict correctness. It is a multi-step evaluation system based on SOC research and on what the industry considers important for SOC

reports. The evaluation loop receives the generated SOC report paired with the alert details that the report is based on. A checklist of relevant aspects of a SOC report, synthesized by Okada et al. [30], is compared with the alert details. The checklist selection gate outputs the list of checklist items deemed relevant to analyze. MESSALA then uses a two-part multi-perspective reconciliation pipeline: a high-level evaluation and an in-depth evaluation followed by reconciliation. This reconciliation pipeline is repeated once for each checklist item. For each item, the report is analyzed in a high-level pass that checks whether the report fulfills the requirement in question, such as whether it discusses host behavior. In parallel, an in-depth judge uses a granularization guideline developed by Okada et al. [30] to break the checklist item into checkpoints, which are then evaluated from 1 to 5. The scores from the high-level and in-depth judges are reconciled to give the final score for the checklist item. The checklist-item scores are then averaged to produce the final MESSALA score for the report.

MESSALA was chosen because it provided all the evaluation prompts used in the framework, making it easy to apply to this project. MESSALA is paired with the F1-based decision-outcome metric rather than used as a replacement for it.

Figure 4.5 summarizes the construction of the MESSALA knowledge base from the source material shown in the figure: Kersten et al.'s four-stage analyst process, own SOC-practitioner interviews, and IT/OT reporting literature.

Figure 4.6 zooms in on the evaluation stage. MESSALA first selects relevant checklist items c_i from the report and alert metadata, scores each selected item as m_i, and aggregates the item-level scores into $e_M(R)$. In parallel, the report verdict $v(R)$ is extracted as $\hat{y}$ and compared with the hidden ground-truth label y to calculate e_F.

Figure 4.7 further zooms in on the item-level MESSALA scoring step. For each selected checklist item, the high-level perspective produces an initial score, while the in-depth perspective uses the granularization guideline to derive report-specific checkpoints. A final reconciliation step then combines these perspectives into the per-item score, explanation, and feedback used by the aggregation stage.

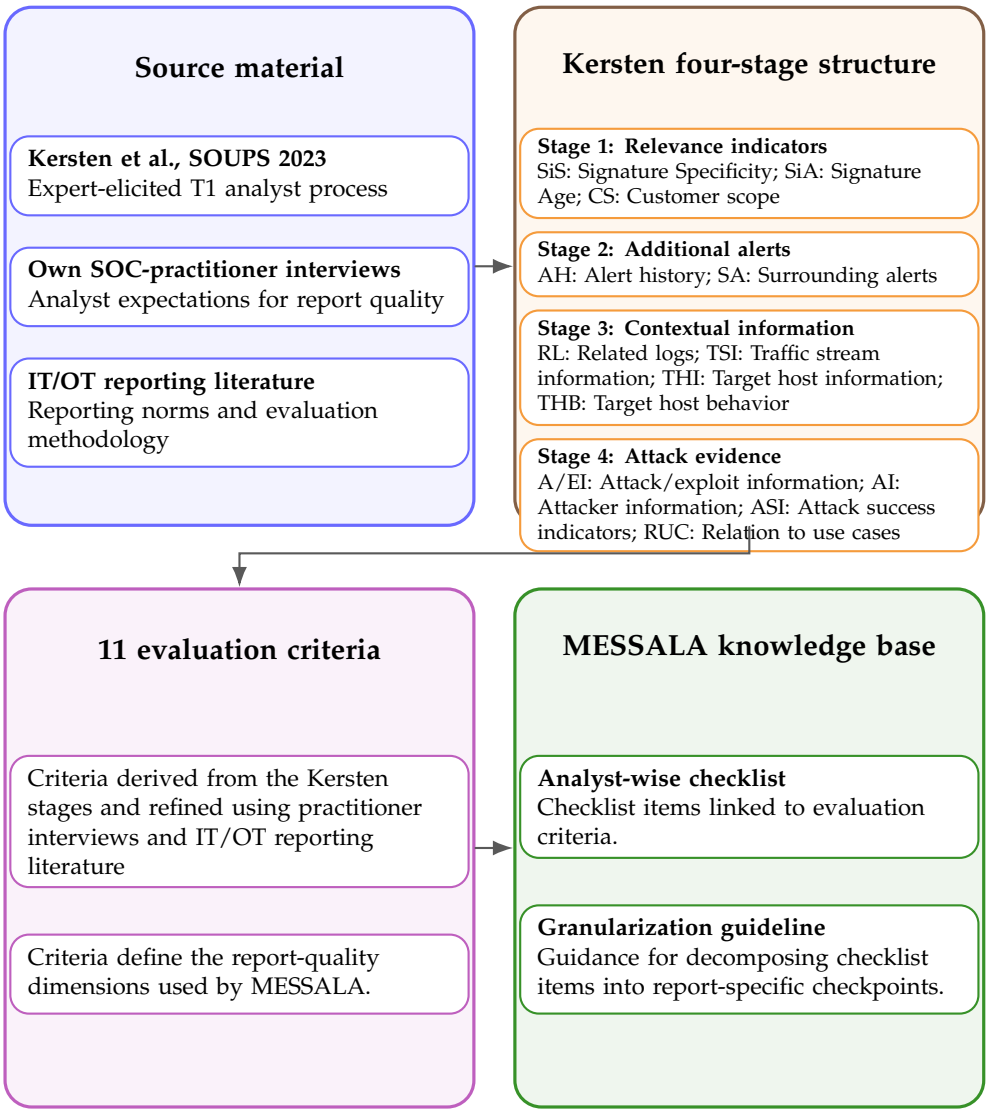


Figure 4.5: Construction of the MESSALA knowledge base from Kersten et al.'s four-stage process, SOC-practitioner interviews, and IT/OT reporting literature.

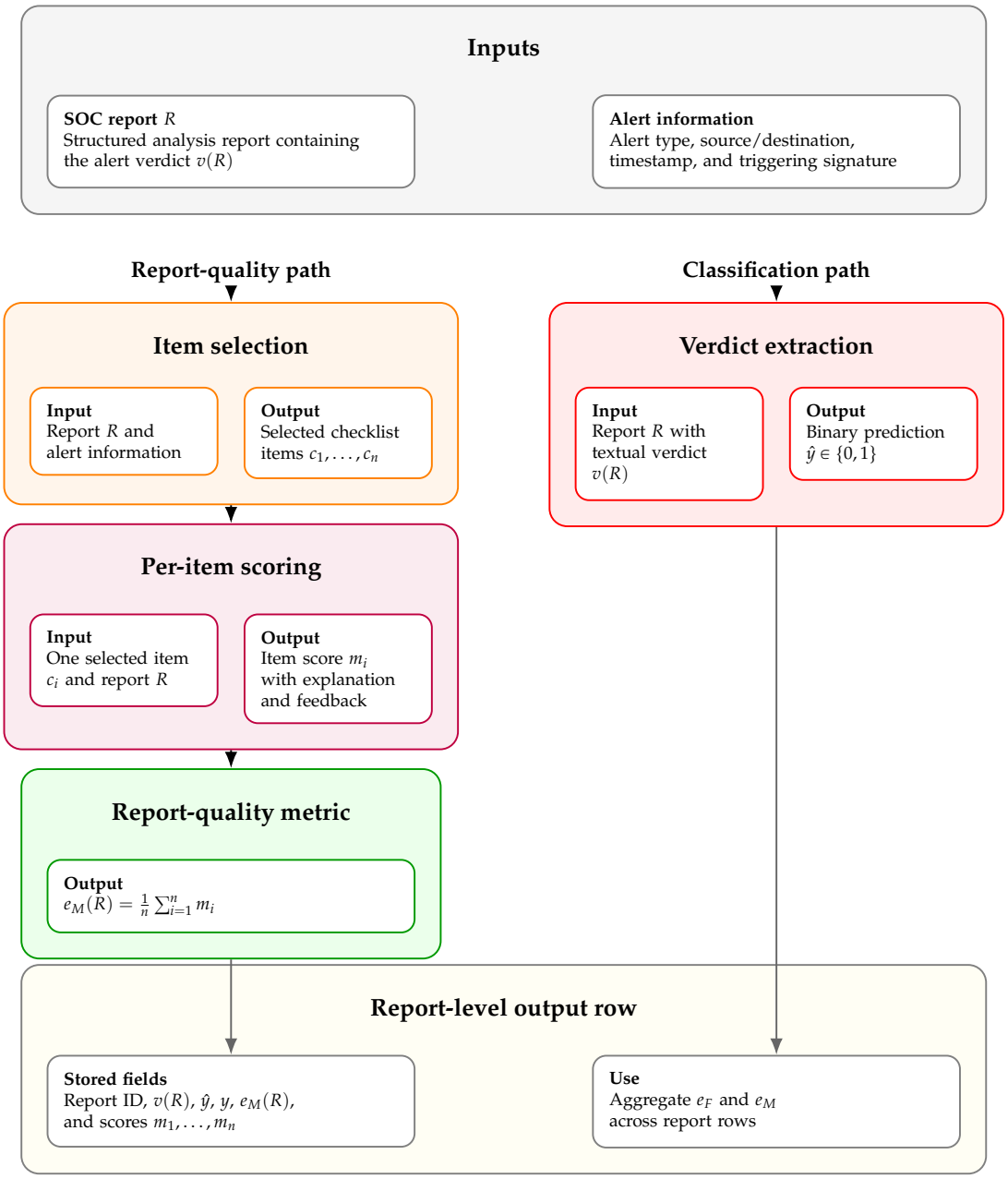


Figure 4.6: Evaluation pipeline for generated alert reports. The report-quality path selects MESSALA checklist items c_i, scores each item as m_i, and aggregates the resulting scores into $e_M(R)$, while the classification path extracts $v(R)$ as $\hat{y}$ and compares it with the ground-truth label y to calculate e_F.

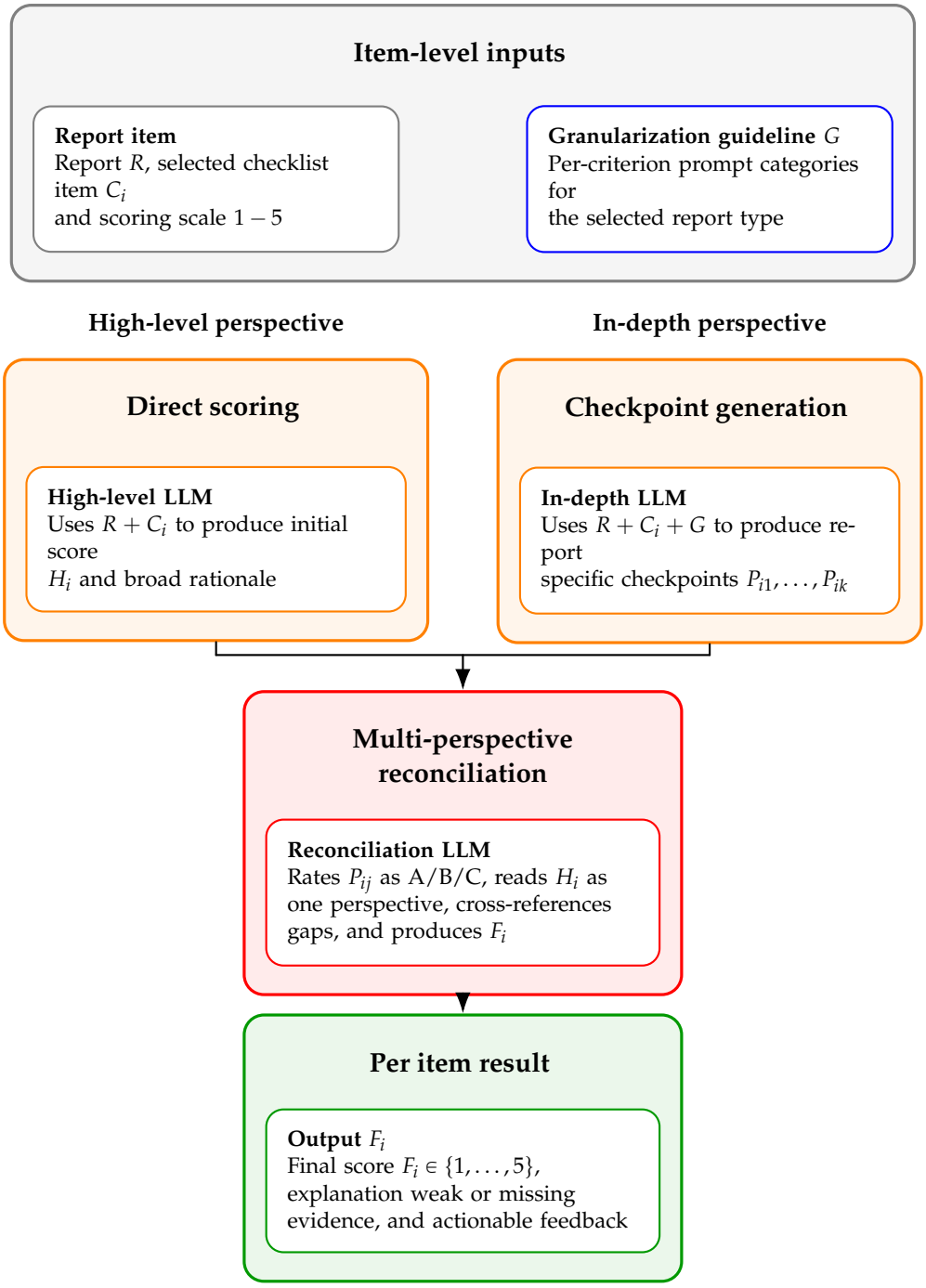


Figure 4.7: Item-level MESSALA scoring using parallel high-level and in-depth perspectives followed by multi-perspective reconciliation. The final per-item result contains the numerical score, explanation, evidence gaps, and actionable feedback returned to the report-level evaluation.

Example MESSALA output. The input to MESSALA in this thesis is the generated alert report together with the alert details. The output contains an aggregate report-quality score, the selected checklist items, item-level scores, explanations, evidence gaps, and actionable feedback. Listing 4.1 shows an example of this output structure.

Listing 4.1: Example MESSALA output for one generated alert report.

```
{
  "report_id": "alert_042",
  "messala_score": 3.7,
  "selected_checklist_items": [
    {
      "item_id": "evidence_grounding",
      "score": 4,
      "explanation": "The report cites the focal alert, command line, user, host, and related
          process evidence. The evidence is mostly tied to the final verdict.",
      "evidence_gaps": [
        "The report does not clearly distinguish focal-alert evidence from broader background
            context."
      ],
      "actionable_feedback": "State explicitly which evidence belongs to the focal alert and
          which evidence is only surrounding context."
    },
    {
      "item_id": "triage_reasoning",
      "score": 3,
      "explanation": "The report explains why the activity may be benign, but the reasoning is
          somewhat compressed and does not fully compare alternative explanations.",
      "evidence_gaps": [
        "No explicit discussion of why the alert should not be escalated despite suspicious
            command-line features."
      ],
      "actionable_feedback": "Add a short benign-versus-malicious comparison before the verdict."
    },
    {
      "item_id": "analyst_actionability",
      "score": 4,
      "explanation": "The report gives a clear verdict and enough context for a Tier-1 analyst to
           understand the decision.",
      "evidence_gaps": [
        "Recommended next steps are minimal."
      ],
      "actionable_feedback": "Add one concrete follow-up, such as checking whether the same user
          or host repeats the activity."
    }
  ],
  "aggregate_explanation": "The report is generally useful and grounded, but loses points because
       it does not always separate direct focal-alert evidence from weak contextual signals.",
  "final_feedback": "Improve the report by explicitly labelling direct evidence, supporting
      context, and uncertainty."
}
```

Evaluation and Final Test Split

To reduce the risk that iterative development overfitted the results, two different alert sets were used. The development evaluation set was used while designing and debugging the base agent, the initial K-MCP agent, and the later K-MCP improvements. The final test set was kept separate and was used only after the selected K-MCP configuration had been frozen. In other words, the workflow was step-by-step development on the evaluation set, followed by a final assessment on the unseen final test set.

The benign part of the development evaluation set was created as described in Section 4.5. After the K-MCP improvements had been selected, a second benign split was created to act as the unseen holdout for final performance assessment. The final test set contained 76 alerts: 59 benign alerts and 17 malicious alerts. The malicious alerts were taken from the public MORDOR 2019 dataset and added as the positive class.

The final verdict metrics are reported using repeated runs where available, rather than treating one selected run as the fixed outcome. MESSALA is used alongside the verdict metrics to evaluate report quality, grounding, and analyst-facing usefulness.

Evaluation workflow

The evaluation notation used in the figures and result tables is summarized as follows:

$$\mathcal{E} = \{e_F, e_M\}$$

where e_F denotes the verdict-correctness metric and e_M denotes the MESSALA report-quality metric. For each generated SOC report R, the textual verdict $v(R)$ is extracted and converted to a binary prediction $\hat{y}$, where 1 denotes malicious and 0 denotes benign. This prediction is compared with the hidden ground-truth label y when computing e_F. For report quality, MESSALA selects checklist items $c_1, \ldots, c_n$, assigns each selected item an item-level score m_i, and aggregates these item scores into the report-level metric $e_M(R)$.

Each evaluated configuration was run on the same alert set and produced one structured SOC report R per alert. Figure 4.8 gives a zoomed-out view of the complete alert-report evaluation workflow. The alert information was visible to the agent, while the ground-truth label y was kept hidden until scoring.

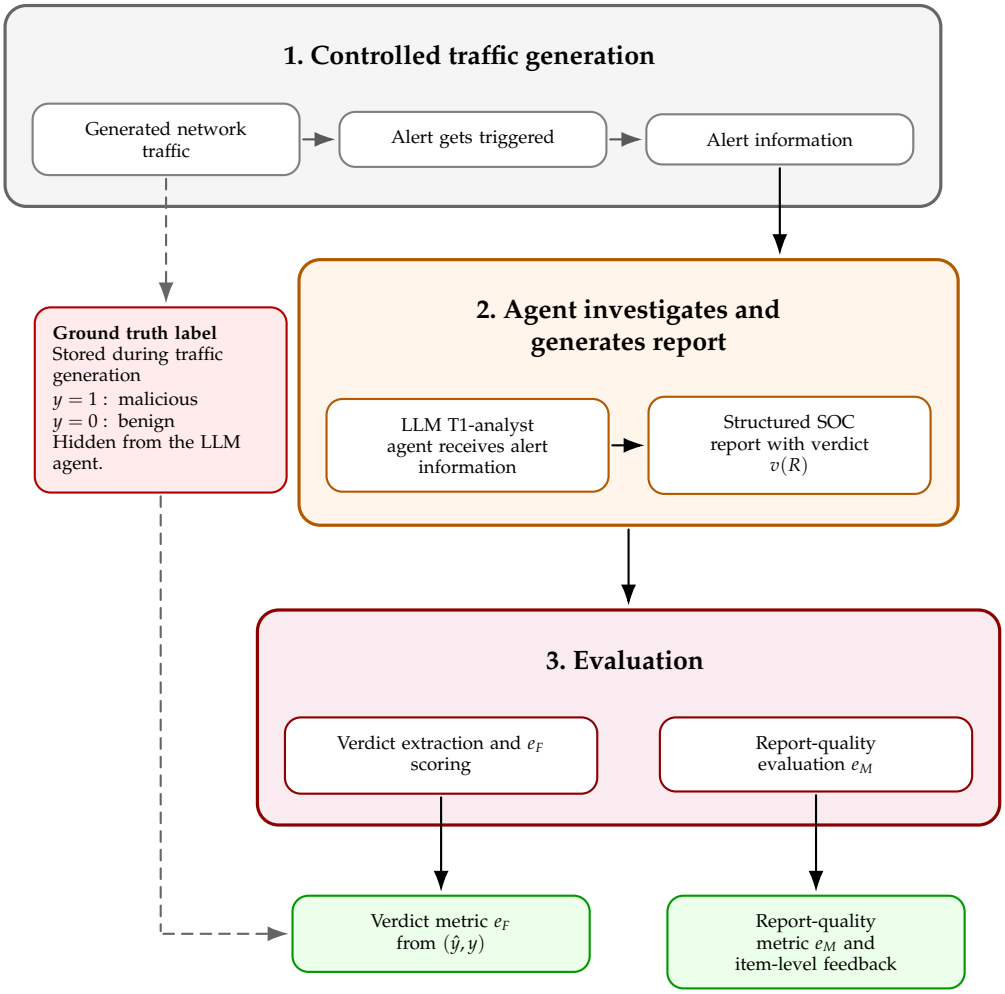


Figure 4.8: Overall alert-report evaluation workflow. Alert information is provided to the LLM agent, while the ground-truth label y remains hidden until the extracted prediction $\hat{y}$ is compared with it for e_F scoring. The report-quality metric e_M is computed separately with MESSALA.

5 Results

The results chapter presents the empirical results of the thesis in relation to the three research questions. It reports how the evaluated pipeline variants performed on the alert analysis task using precision, recall, F1-score, accuracy, and MESSALA. The thesis mainly evaluates whether contextual information helps reduce false positives on benign SOC alerts. The malicious alerts were taken from the public MORDOR 2019 dataset and added as malicious examples for scoring. This makes precision, recall, and F1 more useful than accuracy alone. The chapter also presents the factor-level results and the outcome of the data collection process. The purpose of this chapter is to present the findings clearly, whereas the interpretations are presented in Chapter 6.

5.1 Overview of Results

The results are organized according to the three research questions presented in Chapter 1. First the chapter presents the overall comparison between the evaluated pipeline variants. Second, it reports the factor results and examines how different contextual capabilities affected system performance. Third, it presents the outcome of the data collection process and resulting dataset.

5.2 Results for Research Question 1

The main comparison for this research question is between the base agent and the selected scoped K-MCP configuration. The selected K-MCP configuration excludes AH and SA because repeated evaluation showed that those context types could encourage contextual overreach in this implementation. The main evaluation focus is whether scoped contextual information helps the agent avoid false positives on benign SOC activity. The malicious alerts were taken from the public MORDOR 2019 dataset and used as the positive class. This section reports decision outcome with precision, recall, F1, and accuracy. Because MESSALA scoring was resource-intensive, repeated runs were used to estimate variance in the cheaper verdict metrics, while MESSALA was used once as a final report-quality comparison.

Table 5.1 summarizes the historical development iterations described in the Method chapter. These values are used to show how the configuration was refined before the final evaluation, not as clean repeated-run results. The initial K-MCP comparison performed poorly, while later iterations improved F1 by adding Related logs guidance, introducing progressive disclosure, constraining AH/SA context, and finally selecting the scoped K-MCP configuration without AH/SA. The Δ F1 column reports the change in F1 compared with the previous iteration.

Table 5.1: Historical development runs used to guide final configuration selection.

Configuration	Δ F1	F1
Iteration 1: Initial K-MCP comparison	–	0.390
Iteration 2: Related Logs guidance	0.140	0.530
Iteration 3: Progressive disclosure	0.050	0.580
Iteration 4: Global AH/SA contextual-evidence nudge	0.100	0.680
Iteration 5: Scoped K-MCP without AH/SA	0.234	0.914

Table 5.2 shows the original pre-iteration performance before the later prompt, tool-description, and context-scoping changes were made. In this first version, simply adding K-MCP tools did not improve F1, but rather decreased the performance. The base agent reached F1 = 0.440, while the original K-MCP variant reached F1 = 0.390. The K-MCP version produced fewer false positives, but missed more malicious alerts. This table is used as the starting baseline.

Table 5.2: Classification performance for the original systems.

System	TP	FP	TN	FN	Precision	Recall	F1	Accuracy
Base	11	22	37	6	0.333	0.647	**0.440**	0.632
Base + K-MCP	8	16	43	9	0.333	0.471	**0.390**	0.671

On the 76-alert test set, the base reference result correctly classified 16 of the 17 malicious alerts as malicious, giving recall = 0.941. It produced 12 false positives, resulting in precision = 0.572, F1 = 0.711 $\pm$ 0.016, and accuracy = 0.829. Across three clean complete same-configuration reruns, the selected scoped K-MCP configuration also classified 16 of the 17 malicious alerts correctly, so malicious recall remained 0.941. The difference was false positives on benign alerts: the selected scoped K-MCP configuration produced an average of 1.0 false positives, resulting in mean precision = 0.943, mean F1 = 0.942, and mean accuracy = 0.974. Table 5.3 shows the result of this comparison on the 76-alert final test set. The positive class is the malicious verdict.

Table 5.3: Base agent and selected scoped K-MCP performance on the 76-alert evaluation set.

System Variant	TP	FP	TN	FN	Precision	Recall	F1	Accuracy
Base (New Base)	16	12	47	1	0.572	0.941	0.711 ± 0.016	0.829
Final (Scoped K-MCP)	16	1	58	1	0.943	0.941	0.942 ± 0.028	0.974

The selected scoped K-MCP result averages three clean complete same-configuration reruns. The observed F1 range was 0.914–0.970. The improvement over the base reference result was mainly caused by fewer false positives, while malicious recall remained unchanged.

Although all three runs used the same selected configuration, Table 5.4 shows that the number of false positives varied between 0 and 2.

Table 5.5 shows a qualitative report-level example from one benign scheduled-task alert that was handled differently by the base agent and the selected scoped K-MCP configuration. The

Table 5.4: Clean repeated runs of the selected scoped K-MCP configuration.

Run	TP	FP	TN	FN	Precision	Recall	F1	Accuracy
noahsa__rep3	16	2	57	1	0.889	0.941	0.914	0.961
noahsa__rep4	16	0	59	1	1.000	0.941	0.970	0.987
noahsa__rep5	16	1	58	1	0.941	0.941	0.941	0.974
Mean	16.0	1.0	58.0	1.0	0.943	0.941	0.942	0.974

example is included to make the false-positive reduction more concrete: the focal artifact was similar in both reports, but the selected scoped K-MCP report used surrounding host activity to interpret the alert as part of a coherent administrative or testing sequence.

Table 5.5: Qualitative comparison of two reports for the same benign scheduled-task alert.

Aspect	Base agent report	Selected scoped K-MCP report
Outcome	Classified the benign alert as malicious, producing a false positive; MESSALA $e_M = 2.81$.	Classified the same benign alert as benign, producing a true negative; MESSALA $e_M = 4.06$.
Focal evidence	Emphasized `schtasks.exe`, the `WeeklyInventoryExport` task, `whoami /groups`, public-path output, and forced task creation as suspicious focal indicators.	Described the same scheduled-task evidence, but treated it as insufficient on its own to prove malicious activity.
Context use	Stated that no corroborating timeline or administrative change context was available, so the verdict relied mainly on the suspicious focal artifact.	Used pre- and post-alert activity on `pc103`: `maintenance.ps1` queried tasks, created and reviewed the same task, and inspected Task Scheduler logs.
Reasoning pattern	Interpreted the mismatch between an inventory-like task name and a privilege-enumeration command as reconnaissance or disguised malicious tasking.	Interpreted the event as part of a coherent administrative or test workflow and noted that surrounding host alert volume was weak context, not proof.
Uncertainty handling	Acknowledged uncertainty, but still escalated because the focal artifact resembled known attacker behavior.	Reached a benign verdict while still flagging the need to confirm script ownership and task legitimacy.

Tables 5.6, 5.7, and 5.8 present the same example in a more compact form. The purpose is to show the substantive difference between the two generated reports without reproducing the full raw MESSALA output in the results chapter.

Table 5.6: Formatted summary of the good and bad report examples for the same benign alert.

Field	Selected scoped K-MCP report	Base agent report
Configuration	Scoped K-MCP	Base agent
Ground truth	Benign	Benign
Agent verdict	Benign, true negative	Malicious, false positive
MESSALA score	4.06 / 5	2.81 / 5
Core interpretation	The scheduled-task alert was interpreted as part of a coherent administrative or testing sequence on `pc103`.	The scheduled-task alert was interpreted mainly from the suspicious focal artifact and assessed as likely reconnaissance or disguised malicious tasking.
Context use	Used surrounding host activity, including task queries, task creation, task review, and Task Scheduler log inspection.	Stated that no corroborating timeline or administrative change context was available, so the assessment relied mainly on the alert artifact.
Main limitation	The report still required confirmation of script ownership and task legitimacy.	The report acknowledged uncertainty, but escalated because the focal artifact resembled known attacker behaviour.

Table 5.7: Selected report excerpts from the good and bad examples.

Selected scoped K-MCP report excerpt	Base agent report excerpt
The report assessed the focal alert as more likely benign than malicious. It connected the `schtasks.exe` activity to a sequence where `maintenance.ps1` queried scheduled tasks, created `WeeklyInventoryExport`, reviewed the task, and inspected Task Scheduler logs. It also stated that surrounding host alert volume was weak context and should not be used as proof.	The report assessed the same alert as more likely malicious. It emphasized that `WeeklyInventoryExport` executed `whoami /groups` and wrote output to a public path. The mismatch between the inventory-like task name and the privilege-enumeration command was interpreted as reconnaissance or disguised malicious tasking.

Table 5.8: Selected MESSALA feedback for the good and bad report examples.

Item	Focus	Selected scoped K-MCP report	Base agent report
[1]	Basic alert information	Fully met: event type, command details, timing/sequence context, host, and user context were present.	Mostly present: the main activity and host were described, but timing and context were less explicit.
[5]	Main message and rationale	Fully met: the report gave concrete details and explained why the activity was more consistent with administrative or testing behaviour.	Fully met: the report gave a coherent maliciousness rationale based on the suspicious task mechanics.
[12]	Historical/baseline comparison	Partially met: some historical comparison was included, but prior findings and a robust benign baseline were not fully established.	Weakly satisfied: the report did not meaningfully compare the event with prior alerts or normal baseline behaviour.
[21]	Root-cause distinction	Mostly met: the report distinguished likely human/admin or test automation from malicious persistence, but lacked formal operational evidence.	Weakly satisfied: the report focused on suspicious mechanics but did not establish a site-specific human, system, or operational root cause.
[53]	Evidence support and provenance	Mostly satisfied: multiple correlated observations supported the conclusion, but evidence provenance was incomplete.	Weakly satisfied: useful observations were present, but the source and acquisition method for the evidence were not clearly identified.

5.3 Results for Research Question 2

The impact of the Kersten-derived factors was examined through an ablation study. Each K-MCP variant removed or scoped a specific part of the contextual toolset, and the resulting outputs were compared on the same 76-alert test set. Table 5.9 summarizes the repeated ablation-condition results. The selected scoped K-MCP configuration without AH/SA is used as the headline condition because it had the strongest repeated-run performance and the lowest mean false-positive count.

Table 5.9: Repeated ablation-condition summary.

Condition	Runs	F1 mean $\pm$ SD	F1 range	Precision	Recall	Accuracy	Mean FP
K-MCP–SA	3	0.851 ± 0.035	0.821–0.889	0.777	0.941	0.925	4.7
K-MCP–TSI	3	0.719 ± 0.057	0.667–0.780	0.585	0.941	0.833	11.7
K-MCP–SiA	3	0.702 ± 0.031	0.667–0.727	0.560	0.941	0.820	12.7
K-MCP–SiS	3	0.701 ± 0.009	0.696–0.711	0.558	0.941	0.820	12.7
Full K-MCP	3	0.694 ± 0.059	0.653–0.762	0.552	0.941	0.811	13.3
K-MCP–RL	3	0.697 ± 0.030	0.667–0.727	0.553	0.941	0.816	13.0
K-MCP–AH	3	0.687 ± 0.030	0.653–0.711	0.541	0.941	0.807	13.7
K-MCP–AI	3	0.682 ± 0.037	0.640–0.711	0.536	0.941	0.803	14.0
K-MCP–THB	3	0.645 ± 0.027	0.615–0.667	0.491	0.941	0.768	16.7

The repeated ablation-condition summary does not support a simple "more context is better" interpretation. The full K-MCP condition was not the strongest repeated condition. Instead, the selected scoped configuration without AH and SA had the highest mean F1 and lowest mean false-positive count. This suggests that some contextual factors can introduce misleading associations unless carefully constrained.

For factor-level interpretation, the repeated ablation summary in Table 5.9 is used directly rather than converted into a separate ranking table. This avoids treating small or configuration-dependent differences as a precise factor ordering. Among the leave-one-

factor-out conditions, removing SA produced the highest mean F1, while removing THB, AI, and AH produced the lowest mean F1 values and the highest false-positive counts. These results indicate that the contextual factors affected the system differently, but they should be interpreted as configuration-level evidence rather than as a stable universal ranking of Kersten factors.

MESSALA was used as a complementary report-quality measure for the generated reports. Table 5.10 reports the MESSALA factor-ablation summary relative to the selected scoped K-MCP condition.

Table 5.10: MESSALA summary relative to the selected scoped K-MCP condition.

Condition	**MESSALA**	**Δ MESSALA**
Selected scoped K-MCP, no AH/SA	3.375	0.000
K-MCP−RL	3.20	-0.175
K-MCP−THB	3.16	-0.215
K-MCP−SiS	3.29	-0.085
K-MCP−AH	3.33	-0.045
K-MCP−SA	3.31	-0.065
K-MCP−SiA	3.39	+0.015
K-MCP−AI	3.42	+0.045
K-MCP−TSI	3.47	+0.095

The MESSALA scores show that the ablation factors affect report quality differently from verdict correctness. The largest qualitative degradation occurs when removing Target host behaviour and Related logs, which are important grounding sources for SOC reasoning in this evaluation. The MESSALA changes were less uniform than the F1 changes, which supports treating verdict correctness and report quality as related but distinct outcomes.

5.4 Results for Research Question 3

The benign data collection run presented in 4.4.3 yielded 149 alerts in total. These alerts were later sampled to remain within the project's budgetary and time constraints. The sampling presented in 4.5 produced 59 benign alerts for the final test set. The 17 malicious alerts were not generated in this data collection. They were taken from the public MORDOR 2019 dataset and added to create a mixed final test set. The final test set contained 76 alerts: 59 benign alerts and 17 malicious alerts. From the benign dataset, all 59 alerts were related to the host pc103, which acted as the administrative computer. This final test set is separate from the development evaluation set used during the iterative K-MCP design process.

Besides the sampled alerts, approximately 11 million logs were collected from pc101, pc102, and pc103. These logs provided background noise for the evaluation, including benign activity from all three hosts. This allowed the agent to work with a noisy SOC-like context rather than only isolated alert records.

The result for RQ3 is not only the final number of alerts, but also a way of creating this type of evaluation data. A controlled benign dataset for LLM-assisted alert triage can be created by assigning explicit roles to hosts and users and generating repeatable benign activity that matches those roles. Additionally collecting both alerts and surrounding background logs, preserving traceability between generated activity, telemetry, and keeping externally sourced

malicious alerts separate from lab-generated benign alerts. In this thesis, these steps produced a feasible test set where the agent had to classify alerts using noisy contextual evidence rather than isolated alert labels.

5.5 Summary of Findings

The results show three main findings. For RQ1, the selected scoped K-MCP configuration improved precision, F1, and accuracy mainly by reducing false positives, while malicious-alert recall stayed the same as for the base agent. For RQ2, the repeated context-variant summaries showed that the final no-AH/SA scoped configuration was more reliable than the full K-MCP variant in this implementation. This supports the conclusion that context must be scoped carefully rather than added indiscriminately. For RQ3, the data collection and sampling process produced 59 benign alerts and approximately 11 million background logs, but the more general result is the dataset-construction procedure: controlled benign activity should be role-based, traceable, surrounded by background telemetry, and sampled transparently when evaluation cost is constrained. The malicious alerts were added separately from the public MORDOR 2019 dataset.

6 Discussion

This chapter interprets the findings presented in Chapter 5, discusses methodological strengths and limitations, and places the work in a wider research and societal context.

6.1 Overview of the Discussion

The discussion is structured around the three research questions presented in the thesis. First, it discusses the overall performance of the proposed pipeline. Second, it examines what the factor-level results imply about the role of contextual information in automated SOC alert analysis. Third, it evaluates the usefulness and limitations of the generated dataset and experimental environment. The chapter then concludes with a methodological discussion and a reflection on the wider ethical and societal context of the work.

6.2 Discussion of Research Question 1

As shown in Table 5.3, the comparison for RQ1 indicates that the selected scoped K-MCP configuration improved the pipeline's ability to separate benign alerts from malicious ones. The improvement should not mainly be interpreted as the agent finding more malicious activity. Both the base reference result and the selected scoped K-MCP result detected 16 of the 17 malicious alerts in the test set, giving the same recall. The malicious alerts were taken from the public MORDOR 2019 dataset and are best understood as a needed malicious class for the evaluation. The main difference was found among the benign alerts.

This final result should be read together with the original pre-iteration result in Table 5.2. In the first version, adding the K-MCP tools did not improve the main F1 outcome: the original K-MCP variant had lower F1 than the base agent, even though it produced fewer false positives and slightly higher accuracy. Its lower F1 was caused by more missed malicious alerts. This means that the thesis result is not that MCP access alone improved triage. Rather, the improvement appeared only after the later iterations changed how contextual information was exposed and scoped.

The false-positive reduction in the final result is important because false positives have a different practical meaning in a SOC than ordinary classification errors in an abstract benchmark. A false positive still has to be reviewed by an analyst, and repeated unnecessary escalations can reduce trust in an automated triage system even if malicious-alert recall is high. From this perspective, the selected scoped K-MCP configuration's strongest contribution in this evaluation was not that it made the pipeline more aggressive, but that it made the pipeline more selective. This suggests that scoped contextual factors helped the agent decide when suspicious-looking evidence was not enough to justify a malicious verdict.

This interpretation is also consistent with the role of context described by Kersten et al., who show that SOC analysts rely on multiple contextual factors when assessing alerts [21, 20]. However, the result also narrows that claim: context must be scoped to the focal alert. In the evaluated alerts, the selected scoped K-MCP configuration appears to have benefited from context when deciding that benign activity should remain benign, rather than when discovering additional malicious cases. This is a narrower but still important effect: the contextual tools helped reduce over-escalation.

A likely explanation is that benign administrative activity in the dataset sometimes contained surface indicators that can look suspicious in isolation, such as alert names associated with attacker techniques. Without additional context, the base agent may have treated these indicators as stronger evidence than they warranted. The selected scoped K-MCP configuration had access to selected factors of analysis, which may have made it easier to assess whether the suspicious-looking evidence was actually supported by the focal-alert context. However, this finding should still be interpreted within the limits of the generated 76-alert test set and the specific K-MCP implementation used in this thesis. It should not be generalized into a claim that adding MCP tools will always improve SOC agents in operational environments.

The report-quality result strengthens this interpretation. MESSALA shows that the contextual variants did not only change verdict correctness, but also changed the quality and support of the generated SOC reports. This is important because a triage assistant should not only reach the right verdict; it should also provide an analyst with a useful explanation of why that verdict was reached.

6.2.1 Qualitative report-level comparison

As shown in Tables 5.5, 5.6, and 5.8, the same benign alert was assessed differently by the two configurations. This provides a concrete example of how contextual evidence affected alert triage in this evaluation. The formatted report comparison also shows how MESSALA judged the quality of the two answers, not only whether the final verdict was correct.

Both outputs are based on the same alert, but the configurations making the assessment are different. The incorrect malicious verdict was made by the base agent, whereas the correct benign verdict was made by the selected scoped K-MCP configuration. This example shows how an agent can reduce analyst workload when it uses relevant context to avoid unnecessary escalation. By extension, this could help reduce the time lost to false positives, as discussed in Section 2.2.

The outputs also show that different sources of information led to different reasoning. The reasoning behind both verdicts is understandable given the information available to each configuration, but only the configuration with relevant contextual information reached the correct verdict. The base agent emphasized suspicious indicators in isolation, such as *schtasks.exe*, *whoami /groups*, and a public output path. The selected scoped K-MCP report instead connected the alert to surrounding maintenance activity on *pc103*, including *maintenance.ps1*, task queries, and Task Scheduler log inspection.

An interesting observation is that the selected scoped K-MCP configuration did not treat the focal alert and its artifacts as inherently harmless. Instead, it treated the suspicious command as weaker evidence once it was placed within a probable benign administrative sequence. This shows recalibration of suspicion rather than simple permissiveness. By contrast, the base configuration linked the task-name and command mismatch to reconnaissance and disguised malicious tasking. This was plausible security reasoning, but in this case it overgeneralized from these indicators without enough local corroboration.

An additional important observation is that the selected scoped K-MCP configuration gave a correct verdict while still stating that script ownership and task legitimacy should be confirmed. This indicates a mature assessment because it acknowledges remaining uncertainty even though the alert was deemed benign.

The selected scoped K-MCP configuration also received a higher MESSALA score than the base agent, indicating that the correct true-negative assessment was accompanied by stronger evidence use and clearer analytical grounding.

This evaluation pair supports the importance of contextual information described in [21, 20] in this specific scenario. It also shows that the selected scoped K-MCP configuration did not become more permissive or careless, but instead recalibrated suspicion of the focal alert based on additional information. By extension, this indicates that scoped contextual information can help reduce false positives by helping the agent distinguish suspicious-looking but benign administrative work from malicious activity.

6.3 Discussion of Research Question 2

The RQ2 ablation results showed that the different Kersten-derived context configurations had different effects on the agent's behaviour in this project's dataset. This means that contextual information should not be treated as one single type of support. The strongest result is not that every additional factor improves performance, but that context selection matters.

This result fits with the observations by Kersten et al., where some factors of analysis were used more frequently by SOC analysts than others [21]. However, the implementation result is more cautious. AH and SA are useful concepts in human triage, but in this implementation they could lead the agent to overinterpret surrounding or historical alerts. This supports a more careful interpretation of Kersten-factor operationalization: factors should not only be made available, but must be exposed in a way that prevents the model from treating weak contextual signals as direct evidence.

Target host behaviour also had a clear effect, especially on MESSALA. Removing this factor produced the largest report-quality degradation, which suggests that host behaviour helped the agent write reports that were more complete or better supported. The verdict metrics and MESSALA scores did not produce the same ordering across conditions, which supports treating verdict correctness and report quality as related but distinct outcomes rather than reducing the result to one universal factor ranking.

The report-level comparison in Figure 6.1 illustrates how removing Related logs changed one concrete report and verdict produced by the agent. The example should be read as an illustrative case of how contextual evidence can change the agent's reasoning, not as proof of a universal factor ranking.

Overall, the RQ2 results support a selective view of contextual tool design. Adding analyst-relevant context was useful, but not every context type should be exposed in the same way. Across the repeated ablation runs, malicious recall was largely stable and the differences

With Related logs (RL)

Suspicious scheduled task creation was observed on `pc103`. The task `HelpdeskCacheCleanup` executed `cmd /c dir C:\Windows` and wrote the output to `C:\Users\Public`.

The command and task name are unusual, but related host activity did not show corroborating malicious process activity, network activity, or persistence behaviour.

The activity is therefore consistent with administrative or helpdesk testing activity rather than confirmed reconnaissance.

No escalation is supported by the surrounding telemetry. The alert is assessed as benign.

MESSALA: **3.710**
Classification: **true negative**
Verdict: **benign**

Without Related logs (RL)

Suspicious scheduled task creation was observed on `pc103`. The task `HelpdeskCacheCleanup` executed `cmd /c dir C:\Windows` and wrote the output to `C:\Users\Public`.

The command and task name are unusual. In the absence of related host context, the mismatch between the maintenance-like task name and the directory-listing action is suspicious.

The public output path may indicate disguised reconnaissance or staging through LOLBins.

The activity warrants escalation as potentially malicious. The alert is assessed as malicious.

MESSALA: **3.290**
Classification: **false positive**
Verdict: **malicious**

Same focal evidence, different context. Removing Related logs changed the interpretation of the scheduled-task activity, turning a benign case from a true negative into a false positive.

Figure 6.1: Report excerpts for the Related logs ablation. The focal scheduled-task evidence is the same in both variants, but removing Related logs changes the interpretation from administrative or helpdesk testing activity to suspicious reconnaissance or staging.

mainly reflected false positives on benign alerts. This means that the K-MCP setup should not be understood only as adding more information. It should be understood as adding scoped information, where the agent is guided to use contextual evidence without letting surrounding context override the focal alert.

6.4 Discussion of Research Question 3

As described in 4.4.3, a non-trivial set of benign alerts together with substantial background noise was produced during the data collection run. This means the lab design was capable of creating material for evaluation. The malicious alerts were not produced by this lab design, but were added from the public MORDOR 2019 dataset. Together, the benign alerts, public malicious alerts, and background logs created a setting where the agent had to operate with more information than an isolated alert record.

An important aspect of this setup was the ability to control the activity and in that way also the output of the dataset. This allowed for traceability throughout the system and generation of alerts. The traceability further enhanced the internal validity of the study since the evaluated variants were compared under the same underlying conditions. Due to the sampling being structured in a way that ensured a variety of alert types, this dataset seems feasible for comparative evaluation.

However, the dataset has clear limitations in terms of realism and scale. The alerts from the benign dataset are strongly skewed towards activity from pc103. This creates an overrepresentation of administrative activity causing alerts compared to normal office-like activity such as word processing, browsing, or meeting-related software. This imbalance could potentially affect how the findings generalize to broader SOC environments where activity is distributed across many users, roles, and systems.

The existence of 11 million logs suggests substantial background noise. However, the environment is still a simplified simulation rather than a live enterprise network. Although the use of timelines with generated events is useful for reproducibility, it limits the capacity to capture the variability, inconsistency, and organizational context of a live enterprise network with real human behavior.

The usefulness of the dataset therefore lies primarily in enabling a controlled experimental comparison, rather than in perfectly reproducing an enterprise SOC. The findings should be interpreted as evidence of relative performance between variants in a controlled environment, not a one-to-one measure of real-world deployment performance.

An important contribution this thesis provides is a controlled benign-alert dataset with substantial background noise. Existing datasets focus heavily on malicious activity, detection benchmarks, and raw logs, but do not provide benign alerts to assess in an established way. This lack of benign datasets makes it difficult to evaluate whether LLMs in this context can distinguish between noisy benign activity and truly malicious alerts. At the same time, the dataset should not be described as broadly representative, since the benign alerts are skewed toward pc103 and the evaluation remains limited in scale.

The contribution is not only the alerts themselves, but also the background noise and simulated benign traffic related to these alerts. This is important because Kersten et al. show that context matters for security analysts [21, 20]. Hence, the dataset contributes an evaluation setting that aligns better with the actual analytical task than many simplified or heavily malicious datasets.

Overall, the results suggest that the environment was appropriate for this thesis' evaluation goals. Future work would need to implement broader host diversity, longer activity periods, and more organizational variation to improve the external validity.

6.5 Methodological Discussion

This thesis used an experimental and comparative design, which was suitable because the goal was to evaluate differences between pipeline variants under controlled conditions. The main strength of this approach was that the same alerts, logs and evaluation procedure were reused across the evaluated variants. This made the comparison more systematic than if each variant had been tested on different material.

The custom lab environment was also a methodological strength. It made it possible to generate traceable alerts together with a large amount of background noise. The use of GHOSTS timelines and Elastic telemetry gave the study a reproducible way to create benign activity under known conditions. However, the environment remained a simulation. This means that it could not fully reproduce the variability, inconsistency and organizational complexity of a live enterprise network.

The generated dataset should therefore be understood as useful for controlled comparison, but limited for generalization. The final test set was small, especially for malicious alerts. This means that some effects, such as whether K-MCP can improve recall on a wider range of malicious activity, may not be visible in the results. The benign alerts were also skewed toward pc103, which acted as the administrative computer. This may have made the benign part of the evaluation less varied than what would be expected in a real SOC environment.

The evaluation metrics also have limitations. F1 was useful for comparing verdict correctness, but it compresses different types of errors into one value. In a SOC setting, false negatives and false positives have different practical consequences. MESSALA was useful for scoring report quality, but it does not guarantee that every factual claim in a report is correct. A report can be structured and well supported according to the metric while still containing incorrect or incomplete evidence. For this reason, the F1 and MESSALA results should be interpreted together rather than as replacements for each other.

Overall, the method gives a reproducible basis for comparing the tested variants, but the external validity is limited. The results should therefore be interpreted as evidence of relative performance in this controlled environment, not as a direct prediction of operational SOC performance. Future work would need a larger and more varied dataset, repeated runs, and preferably evaluation in a more realistic or live SOC-like setting.

6.6 The Work in a Wider Context

In a security setting, AI support can increase efficiency, but it also introduces the risk that the system produces plausible-sounding but incorrect interpretations of alerts. This means that the system introduced in this thesis is better understood as decision support rather than as a replacement for the human analyst, especially in cases where alerts are ambiguous or context-dependent. Responsibility for final decisions must remain with the human analyst or the organization deploying the system, since the model itself cannot be held accountable for incorrect assessments.

SOC alert analysis relies on logs, host activity, process information and user-related activity, all of which may contain sensitive organizational or personal data. Giving an AI agent access to such data raises the question about how much data should be exposed to the model and

how that should be handled securely. Even in this thesis, the design reflects a certain concern: contextual information is beneficial for analysis, but the use of that context might also increase privacy and confidentiality risks. In practical deployments, organizations would need clear policies for data minimization, access control, and model usage, especially if external or cloud-hosted models were involved. This creates a tension between analytical usefulness and privacy.

A system of this kind has the potential to reduce analyst workload by helping triage alerts, summarize evidence, and surface relevant contextual information. This could aid security analysts in environments where large volumes of background noise and repetitive first-line analysis consume substantial time and attention. At the same time, increased reliance on AI support may change the role of junior analysts, especially if parts of the initial triage process become automated or heavily model-assisted. There is a societal and professional question of whether such systems mainly augment analysts or deskill parts of the profession by reducing direct engagement with raw evidence. From a positive perspective, LLM-supported analysis could improve consistency, reduce time spent on repetitive tasks and allow analysts to focus on complex and judgement-heavy investigations.

6.7 The recurring malicious alert assessed as benign

The recurring false negative was an LSASS module-load alert in which lsass.exe loaded dhcpcsvc6.dll from `C:\Windows\System32`. At the focal-alert level, the event appeared benign. The DLL was Microsoft-signed, located in an expected system path, and lacked direct evidence of tampering or credential dumping. However, the event was labelled malicious because it occurred within a broader Empire/APT3 compromise scenario on the same host. This illustrates a granularity mismatch between alert-local evidence and scenario-level ground truth. The guarded agent reduced false positives by emphasizing focal-alert evidence, but this same discipline made it less likely to classify benign-looking events as malicious when the maliciousness depended on broader campaign context.

6.8 Limitations and Future Work

This section summarizes the main limitations of the study.

6.8.1 Limited dataset size and scope

The evaluation used a small labelled alert set. This was enough for a controlled comparison between configurations, but it is not enough to support broad claims about operational SOC performance. The benign alerts were also strongly shaped by the lab setup and were mostly connected to administrative activity from pc103. This limits how representative they are of a larger enterprise environment.

6.8.2 Malicious-sample provenance

The malicious alerts were not generated in the lab environment. They were taken from the public MORDOR 2019 dataset and were included mainly to prevent the evaluation from rewarding an agent that classifies every alert as benign. Since the malicious samples come from a public dataset, they may also have been present in GPT-5.4's training data. The malicious-alert results should therefore be interpreted carefully.

6.8.3 Controlled environment and limited repetitions

The final selected classification result is based on three clean complete same configuration reruns. This is stronger than a single run, but still limited. MESSALA is reported as one final report-quality comparison because repeating the MESSALA evaluation in the same way as the verdict metrics was outside the resource budget.

6.8.4 No human analyst study

The thesis evaluates report quality with MESSALA rather than through a human analyst study. This makes the evaluation practical and repeatable, but it does not show how real SOC analysts would use, trust, or challenge the generated reports in an operational workflow.

6.9 Summary of the Discussion

The discussion shows that the selected scoped K-MCP configuration mainly improved the evaluated pipeline by reducing false positives. The RQ1 comparison shows a strong verdict-level improvement, but the original pre-iteration result shows that this improvement should not be attributed to MCP tool access alone. MESSALA provides a single final report-quality comparison. The RQ2 condition summaries further show that not all context types contributed equally and that the full K-MCP condition was not the strongest repeated condition. The selected no-AH/SA scoped configuration had the highest clean repeated mean F1 and the lowest mean false-positive count, which supports the argument that contextual evidence must be scoped carefully. The generated dataset made it possible to compare the variants under controlled conditions, but it also limited how far the results can be generalized.

7 Conclusion

This thesis investigated whether an LLM-based pipeline can be designed to support automated SOC alert analysis by contextualizing alert evidence, producing triage reports, and classifying alerts as malicious or benign.

The main finding is that an LLM-based pipeline can be designed and tested in a controlled SOC-like environment, but its performance depends on the quality and scope of the context given to the agent. As shown in Chapter 5, the selected scoped K-MCP configuration, which had no *AH* or *SA*, averaged F1 = 0.942 $\pm$ 0.028 across three clean complete same-configuration reruns on the 76-alert evaluation set, compared with F1 = 0.711 $\pm$ 0.016 for the base reference result. The improvement came mainly from reducing false positives: the base reference result produced 12 false positives, while the selected scoped K-MCP configuration produced an average of 1.0 false positives. Malicious recall did not improve, as both configurations classified 16 of the 17 malicious alerts correctly.

This improvement should be understood in relation to the original pre-iteration baseline. In the first version, the original K-MCP variant performed worse on F1 than the base agent, with F1 = 0.390 compared with F1 = 0.440. Although it reduced false positives and slightly increased accuracy, it missed more malicious alerts. The final result therefore does not show that simply adding K-MCP tools was sufficient. It shows that the later design iterations were necessary for contextual evidence to become useful in this evaluation.

The development process showed that even though Related logs contained important grounding evidence, the agent did not use them sufficiently. This led to clearer instructions about how Related logs were supposed to be used. Contextual information also had to be managed through progressive disclosure so that the agent could start from compact summaries and only inspect detailed evidence when needed. The initial MCP configuration returned full verbose outputs for every tool call, for example multiple full surrounding-alert JSON objects when querying for *SA*. This motivated progressive disclosure of tool outputs by letting the agent successively gather more information about each factor.

Qualitative report inspection and repeated evaluation showed that AH and SA could be misused in a guilt-by-association way. The final selected configuration therefore removed AH and SA rather than treating the full context configuration as the headline result.

The analysis of contextual configurations shows that simply adding more information does not always help. Historical and single-run experiments suggested that Related logs, Target host behaviour, and Signature specificity could have clear effects, but the clean repeated-run summaries showed a stronger design lesson: context must be scoped carefully. Across the reported runs, malicious recall stayed largely stable. The main effect was instead that scoped context reduced false positives. This suggests that K-MCP context mainly helped the agent distinguish benign administrative activity from genuinely suspicious behaviour.

The thesis also finds that building suitable datasets is a major challenge when evaluating LLM-based SOC triage. Many current datasets focus on attacks, raw logs, or detection labels, but do not provide benign alerts with realistic background noise. To address this, the thesis generated a controlled benign alert dataset in the lab environment. The malicious alerts were not generated in the lab. They were taken from the public MORDOR 2019 dataset and added so that the evaluation would not reward an agent that classifies every alert as benign.

A controlled benign dataset for LLM-assisted alert triage can be built in an isolated and monitored environment, use role-defined hosts and users, generate repeatable benign activity that is plausible for those roles, collect both alerts and surrounding background logs in order preserve traceability from generated activity to alert evidence. This approach makes the dataset useful for evaluating whether an agent can reason over noisy benign context, while still making the limits of realism and scale explicit.

The main contributions of this thesis are the design of an LLM-assisted SOC alert analysis pipeline, the evaluation of contextual factors based on Kersten's work [21, 20], and the creation of a benign-alert evaluation setup with background noise. Together, these contributions show that LLM agents can be studied as tools to support analysts, not solely as standalone classifiers.

However, these results should be viewed with caution. The evaluation set is small, the benign alerts are mostly connected to pc103, and each version was tested with limited repetitions. The malicious samples come from public MORDOR 2019 data and may have been present in GPT-5.4's training data. The malicious-alert result should therefore not be overinterpreted as evidence of general attack-detection capability. Also, the lab environment does not fully reflect the variety and complexity of a real SOC. This means that the findings show a pattern in a controlled setting, especially for false-positive reduction, but do not prove how the pipeline would work in an actual SOC environment.

The main conclusion is therefore not that more context always improves LLM-assisted SOC analysis. Rather, carefully scoped contextual access can improve false-positive handling while preserving malicious recall. In this implementation, alert-history and surrounding-alert context could encourage over-attribution when not scoped carefully. This supports the thesis' framing of LLM agents as evidence-guided decision-support systems for SOC triage, not as autonomous production-ready classifiers. The false-positive reduction can help in reducing alert fatigue. Future research needs to test the pipeline on larger and more varied datasets, run repeated tests to check stability, and examine how analysts use these reports in real workflows.

Bibliography

[1] Bushra A. Alahmadi, Louise Axon, and Ivan Martinovic. "99% False Positives: A Qualitative Study of SOC Analysts' Perspectives on Security Alarms". In: *31st USENIX Security Symposium (USENIX Security 22)*. Boston, MA: USENIX Association, Aug. 2022, pp. 2783–2800. ISBN: 978-1-939133-31-1. URL: https://www.usenix.org/conference/usenixsecurity22/presentation/alahmadi.

[2] M. Anandha Meenachi, Deepa Karuppaiah, and Saravanakumar Kandasamy. "A Comparative Analysis of Prominent Operating Systems: Windows, Android, Linux, and Others". In: *ICT for Intelligent Systems*. Ed. by Jyoti Choudrie, Parikshit N. Mahalle, Thinagaran Perumal, and Amit Joshi. Singapore: Springer Nature Singapore, 2026, pp. 581–593.

[3] Daman Arora and Andrea Zanette. *Training Language Models to Reason Efficiently*. 2025. URL: https://arxiv.org/abs/2502.04463.

[4] Rob Barrett, Eser Kandogan, Paul P. Maglio, Eben M. Haber, Leila A. Takayama, and Madhu Prabaker. "Field studies of computer system administrators: analysis of system management tools and practices". In: *Proceedings of the 2004 ACM Conference on Computer Supported Cooperative Work*. CSCW '04. Chicago, Illinois, USA: Association for Computing Machinery, 2004, pp. 388–395. ISBN: 1581138105. DOI: 10.1145/1031607.1031672. URL: https://doi-org.e.bibl.liu.se/10.1145/1031607.1031672.

[5] Mohan Baruwal Chhetri, Shahroz Tariq, Ronal Singh, Fatemeh Jalalvand, Cecile Paris, and Surya Nepal. "Towards Human-AI Teaming to Mitigate Alert Fatigue in Security Operations Centres". In: *ACM Trans. Internet Technol.* 24.3 (July 2024), 12:1–12:22. ISSN: 1533-5399. DOI: 10.1145/3670009. URL: https://dl.acm.org/doi/10.1145/3670009 (visited on 12/15/2025).

[6] Farid Binbeshr, Muhammad Imam, Mustafa Ghaleb, Mosab Hamdan, Mussadiq Abdul Rahim, and Mohammad Hammoudeh. "The Rise of Cognitive SOCs: A Systematic Literature Review on AI Approaches". In: *IEEE Open Journal of the Computer Society* 6 (2025), pp. 360–379. DOI: 10.1109/OJCS.2025.3536800.

[7] Guiming Hardy Chen, Shunian Chen, Ziche Liu, Feng Jiang, and Benyou Wang. *Humans or LLMs as the Judge? A Study on Judgement Biases*. 2024. URL: https://arxiv.org/abs/2402.10669.

[8] Saurabh Chhajed. *Learning ELK stack*. Packt Publishing Ltd, 2015.

[9] Adrien Defer. "Generating realistic background traffic using GHOSTS framework". Master's thesis. Universite Libre de Bruxelles, 2023. URL: https://cylab.be/publications/51/2023-generating-realistic-background-traffic-using-ghosts-framework.

[10] Elastic. *Elastic Defend Integration*. 2025. URL: https://www.elastic.co/docs/reference/integrations/endpoint (visited on 06/15/2026).

[11] Elastic. *Elastic Licensing FAQ*. https://www.elastic.co/pricing/faq/licensing. Accessed: 2025-01-15. 2024.

[12] Elastic. *Kibana: Visualize, explore, and manage data in Elasticsearch*. 2025. URL: https://www.elastic.co/kibana (visited on 12/10/2025).

[13] Elastic. *Logstash Reference*. Version 8.19. Accessed: 2025-12-10. 2025. URL: https://www.elastic.co/docs/reference/logstash.

[14] Gustavo González-Granadillo, Susana González-Zarzosa, and Francesco Faiella. "Security Information and Event Management (SIEM): Analysis, Trends, and Usage in Critical Infrastructures". In: *Sensors* 21.14 (2021), p. 4759. DOI: 10.3390/s21144759. URL: https://doi.org/10.3390/s21144759.

[15] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning". In: *arXiv preprint arXiv:2501.12948* (2025).

[16] Ismayil Hasanov, Seppo Virtanen, Antti Hakkala, and Jouni Isoaho. "Application of Large Language Models in Cybersecurity: A Systematic Literature Review". In: *IEEE Access* 12 (2024), pp. 176751–176778. DOI: 10.1109/ACCESS.2024.3505983.

[17] Haibo He and Edwardo A. Garcia. "Learning from Imbalanced Data". In: *IEEE Transactions on Knowledge and Data Engineering* 21.9 (2009), pp. 1263–1284. DOI: 10.1109/TKDE.2008.239.

[18] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. "Model context protocol (mcp): Landscape, security threats, and future research directions". In: *ACM Transactions on Software Engineering and Methodology* (2025).

[19] Egil Karlsen, Xiao Luo, Nur Zincir-Heywood, and Malcolm Heywood. "Benchmarking Large Language Models for Log Analysis, Security, and Interpretation". In: *J. Netw. Syst. Manage.* 32.3 (June 2024). ISSN: 1064-7570. DOI: 10.1007/s10922-024-09831-x. URL: https://doi.org/10.1007/s10922-024-09831-x.

[20] Leon Kersten, Kim Beelen, Emmanuele Zambon, Chris Snijders, and Luca Allodi. "A Field Study to Uncover and a Tool to Support the Alert Investigation Process of Tier-1 Analysts". In: *Proc. of USEC* (2025), pp. 1–15.

[21] Leon Kersten, Tom Mulders, Emmanuele Zambon, Chris Snijders, and Luca Allodi. "'give me structure': Synthesis and evaluation of a (network) threat analysis process supporting tier 1 investigations in a security operation center". In: *Nineteenth Symposium on Usable Privacy and Security (SOUPS 2023)*. 2023, pp. 97–111.

[22] Tom Krantz and Alexandra Jonker. *alert fatigue*. May 2025. URL: https://www.ibm.com/think/topics/alert-fatigue.

[23] Max Landauer, Florian Skopik, Markus Wurzenberger, Teodor Sommestad, and Henrik Karlzén. "Benign User Activities that Trigger False Positives in Intrusion Detection Systems: An Expert Survey". In: *Availability, Reliability and Security*. Ed. by Bart Coppens, Bruno Volckaert, Vincent Naessens, and Bjorn De Sutter. Cham: Springer Nature Switzerland, 2025, pp. 25–43. ISBN: 978-3-032-00633-2.

[24] LangChain. *LangChain Documentation*. 2026. URL: https://python.langchain.com/docs/introduction/ (visited on 04/02/2026).

[25] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. "API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs". In: *arXiv preprint arXiv:2304.08244* (2023). DOI: 10.48550/arXiv.2304.08244. URL: https://arxiv.org/abs/2304.08244.

[26] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, et al. "Holistic Evaluation of Language Models". In: *Transactions on Machine Learning Research* (2023). URL: https://openreview.net/forum?id=iO4LZibEqW.

[27] Tianyang Lin, Yuxin Wang, Xiangyang Liu, and Xipeng Qiu. "A survey of transformers". In: *AI Open* 3 (2022), pp. 111–132. ISSN: 2666-6510. DOI: https://doi.org/10.1016/j.aiopen.2022.10.001. URL: https://www.sciencedirect.com/science/article/pii/S2666651022000146.

[28] Mamoru Mimura and Yui Tajiri. "Static detection of malicious PowerShell based on word embeddings". In: *Internet of Things* 15 (2021), p. 100404. ISSN: 2542-6605. DOI: https://doi.org/10.1016/j.iot.2021.100404. URL: https://www.sciencedirect.com/science/article/pii/S2542660521000482.

[29] Javier Navarro, Arnaud Deruyver, and Pierre Parrend. "OMMA: Open Architecture for Operator-Guided Monitoring of Multi-Step Attacks". In: *Journal of Internet Services and Applications* 9.1 (2018), pp. 1–16. DOI: 10.1186/s13174-018-0091-y. URL: https://doi.org/10.1186/s13174-018-0091-y.

[30] Hiroyuki Okada, Tatsumi Oba, and Naoto Yanai. "LLMs, You Can Evaluate It! Design of Multi-perspective Report Evaluation for Security Operation Centers". In: *ArXiv* abs/2601.03013 (2026). URL: https://api.semanticscholar.org/CorpusID:284512501.

[31] Shruti Patil, Vijayakumar Varadarajan, Siddiqui Mohd Mazhar, Abdulwodood Sahibzada, Nihal Ahmed, Onkar Sinha, Satish Kumar, Kailash Shaw, and Ketan Kotecha. "Explainable Artificial Intelligence for Intrusion Detection System". en. In: *Electronics* 11.19 (Sept. 2022), p. 3079. ISSN: 2079-9292. DOI: 10.3390/electronics11193079. URL: https://www.mdpi.com/2079-9292/11/19/3079 (visited on 12/15/2025).

[32] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. "ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs". In: *arXiv preprint arXiv:2307.16789* (2023). DOI: 10.48550/arXiv.2307.16789. URL: https://arxiv.org/abs/2307.16789.

[33] Abhilasha Ravichander, Jillian Fisher, Taylor Sorensen, Ximing Lu, Maria Antoniak, Bill Yuchen Lin, Niloofar Mireshghallah, Chandra Bhagavatula, and Yejin Choi. "Information-Guided Identification of Training Data Imprint in (Proprietary) Large Language Models". In: *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Albuquerque, New Mexico: Association for Computational Linguistics, 2025, pp. 1962–1978. DOI: 10.18653/v1/2025.naacl-long.99. URL: https://aclanthology.org/2025.naacl-long.99/.

[34] Maria Rigaki, Ondřej Lukáš, Carlos Catania, and Sebastian Garcia. "Out of the cage: How stochastic parrots win in cyber security environments". In: *Proceedings of the 16th International Conference on Agents and Artificial Intelligence (ICAART)*. Vol. 2. SciTePress. 2024, pp. 774–781.

[35] Sina Sheikholeslami, Moritz Meister, Tianze Wang, Amir H. Payberah, Vladimir Vlassov, and Jim Dowling. "AutoAblation: Automated Parallel Ablation Studies for Deep Learning". In: *Proceedings of the 1st Workshop on Machine Learning and Systems*. EuroMLSys '21. Online, United Kingdom: Association for Computing Machinery, 2021, pp. 55–61. ISBN: 9781450382984. DOI: 10.1145/3437984.3458834. URL: https://doi-org.e.bibl.liu.se/10.1145/3437984.3458834.

[36] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H. Chi, Nathanael Schärli, and Denny Zhou. "Large Language Models Can Be Easily Distracted by Irrelevant Context". In: *Proceedings of the 40th International Conference on Machine Learning*. Ed. by Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett. Vol. 202. Proceedings of Machine Learning Research. PMLR, July 2023, pp. 31210–31227. URL: https://proceedings.mlr.press/v202/shi23a.html.

[37] Zhengliang Shi, Yuhan Wang, Lingyong Yan, Pengjie Ren, Shuaiqiang Wang, Dawei Yin, and Zhaochun Ren. "Retrieval Models Aren't Tool-Savvy: Benchmarking Tool Retrieval for Large Language Models". In: *Findings of the Association for Computational Linguistics: ACL 2025*. 2025, pp. 24497–24524. URL: https://aclanthology.org/2025.findings-acl.1258/.

[38] Sung Jun Son and Youngmi Kwon. "Performance of ELK stack and commercial system in security log analysis". In: *2017 IEEE 13th Malaysia international conference on communications (MICC)*. IEEE. 2017, pp. 187–190.

[39] Shahroz Tariq, Mohan Baruwal Chhetri, Surya Nepal, and Cecile Paris. "Alert Fatigue in Security Operations Centres: Research Challenges and Opportunities". In: *ACM Comput. Surv.* 57.9 (Apr. 2025). ISSN: 0360-0300. DOI: 10.1145/3723158. URL: https://doi.org/10.1145/3723158.

[40] *Tools — Model Context Protocol Specification*. 2025. URL: https://modelcontextprotocol.io/specification/2025-11-25/server/tools (visited on 04/10/2026).

[41] Dustin Updyke, Geoffrey Dobson, Thomas Podnar, Luke Osterritter, Benjamin Earl, and Adam Cerini. *GHOSTS in the Machine: A Framework for Cyber-Warfare Exercise NPC Simulation*. 2018. URL: https://apps.dtic.mil/sti/trecms/pdf/AD1068374.pdf.

[42] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan Gomez, Łukasz Kaiser, and Illia Polosukhin. *Attention Is All You Need*. June 2017. URL: https://arxiv.org/pdf/1706.03762.

[43] Manfred Vielberth, Fabian Böhm, Ines Fichtinger, and Günther Pernul. "Security Operations Center: A Systematic Study and Open Challenges". In: *IEEE Access* 8 (2020), pp. 227756–227779. DOI: 10.1109/ACCESS.2020.3045514.

[44] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. "Chain-of-thought prompting elicits reasoning in large language models". In: *Advances in neural information processing systems* 35 (2022), pp. 24824–24837.

[45] "Windows Security Logging and Monitoring Policy". In: *Windows® Security Monitoring: Scenarios and Patterns*. John Wiley & Sons, Ltd, 2018. Chap. 1, pp. 1–9. ISBN: 9781119390909. DOI: https://doi.org/10.1002/9781119390909.ch1. eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/9781119390909.ch1. URL: https://onlinelibrary.wiley.com/doi/abs/10.1002/9781119390909.ch1.

[46] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. "A survey of large language models". In: *arXiv preprint arXiv:2303.18223* 1.2 (2023), pp. 1–124.

A JSON Scripts Used for Data Generation

This appendix contains the JSON-based scripts used to generate the synthetic activity and supporting data used throughout the thesis. The purpose of including these scripts is to improve transparency and reproducibility by documenting the configurations that drove the data-generation process.

A.0.1 Example: Timeline Configuration for `IT-WS-01`

Listing A.1: Example JSON timeline used for data generation. the TASK_CREATE variables are placeholder for different commands such as seen the last command field.

```
{
 {
  "Status": "Run",
  "TimeLineHandlers": [
    {
      "HandlerType": "Command",
      "Initial": "",
      "UtcTimeOn": "00:00:00",
      "UtcTimeOff": "23:59:59",
      "HandlerArgs": {},
      "Loop": false,
      "ScheduleType": "Other",
      "TimeLineEvents": [
        {
          "Command": "cmd /c schtasks /query /fo LIST /v",
          "CommandArgs": [],
          "DelayAfter": 5000,
          "DelayBefore": 0
        },
        {
          "Command": "{{TASK_CREATE_CMD}}",
          "CommandArgs": [],
          "DelayAfter": 8000,
          "DelayBefore": 4000
```

```
        },
        {
          "Command": "{{TASK_QUERY_CMD}}",
          "CommandArgs": [],
          "DelayAfter": 4000,
          "DelayBefore": 2000
        },
        {
          "Command": "{{TASK_CLEANUP_CMD}}",
          "CommandArgs": [],
          "DelayAfter": 3000,
          "DelayBefore": 1000
        },
        {
          "Command": "{{TASK_POST_CMD}}",
          "CommandArgs": [],
          "DelayAfter": 3000,
          "DelayBefore": 1000
        },
        {
          "Command": "cmd /c powershell -ExecutionPolicy Bypass -Command \"
              Get-WinEvent -LogName 'Microsoft-Windows-TaskScheduler/
              Operational' -MaxEvents 10 | Select-Object TimeCreated, Id,
              ProviderName | Format-Table -AutoSize\"",
          "CommandArgs": [],
          "DelayAfter": 5000,
          "DelayBefore": 3000
        }
      ]
    }
  ]
 }
}
```

Listing A.2: *admin_powershell_inventory.json* partial used for traffic generation.

```
{
 {
  "Status": "Run",
  "TimeLineHandlers": [
    {
  "Status": "Run",
  "TimeLineHandlers": [
    {
      "HandlerType": "Command",
      "Initial": "",
      "UtcTimeOn": "00:00:00",
      "UtcTimeOff": "23:59:59",
      "HandlerArgs": {},
      "Loop": false,
      "ScheduleType": "Other",
      "TimeLineEvents": [
        {
          "Command": "cmd /c powershell -ExecutionPolicy Bypass -Command \"
              $out='{{OUTPUT_FILE}}'; Get-ComputerInfo | Select-Object
              CsName, WindowsVersion, OsBuildNumber | Out-File -FilePath
              $out -Append\"",
          "CommandArgs": [],
```

```
            "DelayAfter": 5000,
            "DelayBefore": 0
          },
          {
            "Command": "cmd /c powershell -ExecutionPolicy Bypass -Command \"
                $out='{{OUTPUT_FILE}}'; Get-Process | Sort-Object CPU -
                Descending | Select-Object -First 15 ProcessName, Id, CPU |
                Out-File -FilePath $out -Append\"",
            "CommandArgs": [],
            "DelayAfter": 5000,
            "DelayBefore": 2000
          },
          {
            "Command": "cmd /c powershell -ExecutionPolicy Bypass -Command \"
                $out='{{OUTPUT_FILE}}'; Get-NetTCPConnection | Select-Object
                -First 20 LocalAddress, LocalPort, RemoteAddress, State | Out
                -File -FilePath $out -Append\"",
            "CommandArgs": [],
            "DelayAfter": 5000,
            "DelayBefore": 2000
          }
        ]
      }
    ]
}
}
```